

CARL VON OSSIETZKY UNIVERSITÄT OLDENBURG
DEPARTMENT OF ENERGY INFORMATICS

Implementation and Design of a Notification and Suggestion System for Power Grid Events as a SCADA extension.

Bachelor Thesis

to reach the academic degree ‘Bachelor of Science’ in major Computer Science in the
Faculty II - Computer Science, Economics and Law at the Department of Energy
Informatics

submitted by

Philip Weber

born on 12.04.2001 in Bremen

First Examiner: Dr.-Ing. Eric Veith
Second Examiner: Arlena Wellßow, M.Sc.

Oldenburg, 20.03.2026

Acknowledgements

I would like to express my sincere gratitude to everyone who supported me throughout my studies and during the completion of this bachelor's thesis.

I would like to thank the ARL team at OFFIS for welcoming me to their weekly meetings and for the collegial support, constructive feedback, and inspiring discussions.

My special thanks goes to my supervisor, Arlena Wellßow (M. Sc.), for her excellent guidance, valuable discussions, and continuous encouragement throughout the project, as well as Dr.-Ing. Eric Veith for his valuable recommendations on the process of writing a thesis and insight into his team and work.

I also wish Arlena continued success, resilience, and all the best for her doctoral studies.

Last but not least and most importantly, I would like to thank my parents, Martina and Werner Weber, for their constant support. Thank you.

Abstract

The increasing complexity of modern smart grids places growing cognitive demands on operators, who must detect and respond to disturbances across large, interconnected power distribution systems as fast as possible. Supervisory Control and Data Acquisition (SCADA) systems are the established platform for grid monitoring and control, yet their standard interfaces offer limited support for the higher-level cognitive processes that underpin effective operator situational awareness (SA).

This thesis presents "Sally", a modular SCADA extension designed to enhance operator SA in smart grid environments. Sally ingests time-series grid data from mosaik-based co-simulation exports (HDF5) or a TimescaleDB instance, evaluates user-defined rules over incoming measurements, and generates contextually rich notifications that are delivered through multiple coordinated graphical interfaces: a SCADA display with an interactive Single-Line Diagram, a Rule Manager for creating and chaining conditional rules, and a Grafana-based observability stack for telemetry inspection. Communication between all components is mediated by a custom event bus built on different ring-buffer variants and instrumented with OpenTelemetry for end-to-end tracing, metrics, and structured logging.

Beyond these quantitative results, the thesis argues that Sally establishes a broader pathway to improved SA through deterministic scenario replay and structured rule creation. Trainers can design repeatable sessions that expose operators to specific fault patterns, while the Rule Manager turns training into a measurable exercise that maps directly to Endsley's comprehension levels. Together, the proxy scores and the training workflow provide converging evidence that Sally supports SA development across all three levels of Endsley's model.

Contents

Nomenclature	ix
1 Introduction	1
1.1 Research Questions	2
2 Foundations	3
2.1 Smart Grids	3
2.2 SCADA Systems	6
2.3 Situational Awareness	8
2.4 HMI Design and Alarm Concepts	11
2.5 Co-Simulation in CPES	13
2.6 Relevant Technologies in Computer Science	15
3 Requirements & Evaluation Design	22
3.1 System Requirements	22
3.2 Evaluation Design	25
4 System Design Concept	28
4.1 GUI Integration Concept	28
4.2 Planned Architecture	31
5 Implementation	33
5.1 Used Technologies	33
5.2 Sally Event-Bus	34
5.3 Co-simulation Framework "mosaik"	39
5.4 Grafana Stack Setup	41
5.5 Architecture Overview	42
5.6 Back-End Architecture Overview	45
5.7 SCADA Orchestration Service	50
5.8 Front-End Architecture Overview	54
6 Experiments and Evaluation	60
6.1 Event Bus Evaluation	60
6.2 Situational Awareness Proxy Evaluation	64
7 Discussion	68
7.1 Interpretation of Results	68

7.2	Limitations	72
7.3	Implications for Real-World SCADA Deployment	74
8	Conclusion & Future work	76
8.1	Conclusion	76
8.2	Future Work	78
A	Appendix	80
A.1	Database	80
A.2	Grafana	81
A.3	Python and Dependencies	84
A.4	Event-Bus	89
	List of Figures	91
	List of Tables	93
	Bibliography	94

Glossary

API Application Programming Interface. 41

BAN Business Area Network. 4, 5

BMWK Federal Ministry for Economic Affairs and Climate Action. 1

CPES Cyber-Physical Energy System. 13

FMI Functional Mock-up Interface. 14

GIL Global Interpreter Lock. 73

GUI Graphical User Interface. 25–29, 31, 33, 41, 64, 65, 76

HAN Home Area Network. 4

HDF5 Hierarchical Data Format version 5. 15, 26–28, 31, 34, 41, 45, 47–49, 51, 54, 57, 64, 66, 72, 74, 79

HMI Human Machine Interface. 3, 7, 9–12, 14, 73, 75, 78, 91

IAN Industrial Area Network. 4

ICT Information and Communication Technology. 3

KRITIS Critical Infrastructure. 1

KSG Klimaschutzgesetz. 1

MPMC Multiple Producer Multiple Consumer. 24, 60–62, 71

MPSC Multiple Producer Single Consumer. 24, 60–62, 71

MsbG Messstellenbetriebsgesetz. 1

NAN Neighbourhood Area Network. 4

NASA-TLX NASA Task Load Index. 9, 26, 72, 78

NREL National Renewable Energy Laboratory. 14

OTEL Open Telemetry. 23, 27, 31, 33, 42, 45, 64, 74, 76, 77, 83

PLC Programmable Logic Controller. 6

RTU Master Terminal Unit. 7

RTU Remote Terminal Unit. 6

SA Situational Awareness. 7–12, 14, 25, 26, 28, 33, 42, 47, 60, 64, 65, 67–73, 76–78, 93

SAGAT Situation Awareness Global Assessment Technique. 9, 26, 72, 78

SALLY Smart-Grid-Ally. 22, 23, 25, 26, 28, 31, 33, 34, 39–42, 45, 46, 50, 54, 55, 57, 60, 64, 66, 68–71, 74–78, 80, 89, 91–93

SCADA Supervisory Control and Data Acquisition. 2, 3, 5–7, 9–12, 22–31, 45, 51, 55, 57–60, 62, 68, 71, 73–77, 79, 91

SLD Single Line Diagram. 9, 11, 26, 29–31, 33, 45, 57–59, 65, 68, 69, 71, 75, 91

SPMC Single Producer Multiple Consumer. 24, 60–62, 71

SPSC Single Producer Single Consumer. 17, 24, 60–62, 71

TTM Telemetry-Timescale-Mqtt-Stack. 34, 42

WAN Wide Area Network. 4

YAML YAML Ain't Markup Language. 33, 41

Nomenclature

General

t	Simulation timestep index
w	Configurable trend-data history window; the number of preceding timesteps required for trend visibility

Perception proxy metrics

$E_{\text{gt}}(t)$	Set of entity-attribute pairs present in the ground-truth data source (HDF5 or TimescaleDB) at timestep t
$E_{\text{gui}}(t)$	Set of entity-attribute pairs actually propagated to the GUI via domain events at timestep t
$C(t)$	Display completeness ratio at timestep t ; defined as $ E_{\text{gui}}(t) \cap E_{\text{gt}}(t) / E_{\text{gt}}(t) $

Comprehension proxy metrics

F_{req}	Set of required contextual fields per notification (entity identifier, measurement attribute, current value, threshold value, severity level, timestamp)
$F_{\text{present}}(n)$	Subset of F_{req} that is non-empty in notification n
$S(n)$	Per-notification structural completeness; defined as $ F_{\text{present}}(n) / F_{\text{req}} $
$\bar{S}$	Scenario-level mean structural completeness over all N notifications in a run
N	Total number of notifications emitted during a scenario replay

1

Introduction

Climate change is one of the most pressing global challenges of our time, with far-reaching consequences for ecosystems, economies, and societies [1]. One part of dealing with this challenge is the energy sector, making its transformation essential for effective climate mitigation strategies, since the energy sector is a direct part of Critical Infrastructure (KRITIS) and impacts climate change in different ways, for example, energy production and energy consumption, as addressed by the book “Climate Change and Future Justice” [2]. Accompanying the challenge of meeting climate goals, Germany introduced a nationwide comprehensive policy initiative called the “Energiewende” [3]. The Energiewende aims to address climate change mitigation across multiple sectors, in part by implementing named energy-sector goals through a smart-meter rollout, which is part of a plan to accelerate digitalization [3]. Central to this effort are legislative measures and the laws that emerged from them, such as the Klimaschutzgesetz (KSG) [4], which sets mandated targets for reducing greenhouse gas emissions, including a sector-specific reduction of 77% by 2030 for the energy sector, as stated in §3 KSG [4].

To achieve the previously mentioned mandated targets, the modernization of traditional power grids into smart grids is a pivotal component of success and contributes to the Energiewende as a whole. The Federal Ministry for Economic Affairs and Climate Action (BMWK) Smart Grid Roadmap [5] outlines a plan for achieving 50% digitalization of distribution grids by 2025, underscoring the importance of smart grid infrastructure in meeting climate goals. To further support the infrastructural goals, the Messstellenbetriebsgesetz (MsbG) [6] mandates a nationwide rollout of smart meters, targeting 80% coverage by 2032. Smart meters are fundamental to smart grids, enabling real-time monitoring of resource consumption (electricity, water, gas), demand response, and the integration of renewable energy sources.

The increasing digitalization of the energy grid will enhance grid efficiency and flexibility by enabling more detailed, frequent data on grid health and energy production and consumption, enabling more efficient distribution [7].

With the digitalization and the aim of a rollout of intelligent measurement systems, as mentioned in [3], comes a larger volume of meter data (electricity, gas, and water consumption data) to process and monitor properly [7].

Because of the importance of processing and monitoring such meter data in smart grids [7] and the planned acceleration of the smart-meter rollout [3], network operators are already confronted with a rising cognitive load [8]. With the increasing and massive amounts of metering data [8], there exists an increased likelihood that network anomalies go undetected,

which, in a worst-case scenario, could result in a complete blackout caused by cascading failures triggered by an initially minor component malfunction or an erroneous meter reading, as evidenced by the severe blackout incident of the Spanish and Portuguese power grids on the 28th of April in 2025 [9].

With this in mind, the need for supportive systems that aid in decision-making in order to reduce the cognitive load of operators in network control rooms has risen in the last years [8].

1.1 RESEARCH QUESTIONS

The central aim of this thesis is to address the challenges of cognitive support for operators in smart grid environments by designing and implementing a supportive system with a visual interface that notifies operators of events and provides suggestions to aid decision-making in tackling grid events. The following research questions will guide the thesis:

Main Research Question

How can a notification and suggestion system be designed and implemented to extract and present critical information, thereby enhancing operator situational awareness in grid scenarios?

Supportive Research Question

What considerations must be taken into account for future integration of the proposed system into an existing Supervisory Control and Data Acquisition (SCADA) system within a network control center, ensuring synchronization between the proposed system and the network control center's technology and data handling for grid state changes and visualization of notifications?

2 | Foundations

This section introduces the foundational concepts and technologies that underpin the design and implementation of an operator support system for smart grid environments. It begins with an overview of smart grids and SCADA systems, which constitute the operational context in which enhanced situational awareness is needed. This is followed by a discussion of Human Machine Interface (HMI) design principles and situational awareness theory, both of which inform the design of effective notification interfaces for grid operators. Subsequently, the key technologies employed during implementation are presented, including Python and the HDF5 data format for data persistence. The section further covers co-simulation as the methodological backbone for reproducible smart grid experimentation, followed by architectural building blocks such as event buses, ring buffers, and time series databases that enable efficient real-time data handling. Finally, general HMI design considerations and the concept of replayability in smart grid simulations are discussed.

2.1 SMART GRIDS

The term *smart grid* describes the integration of digital communication technologies and automated control systems into the traditional power grid in order to create a more efficient, reliable, and flexible energy supply system [10, 11]. Unlike conventional grids, a smart grid combines Information and Communication Technology (ICT) with physical grid components to enable continuous monitoring, control, and optimisation of electricity generation, transmission, distribution, and consumption [12].

This modernisation changes how grid operators supervise and manage the power system. Digital sensing and metering technologies, communication networks, and computational intelligence allow operators to monitor system performance and resource distribution with reduced latency, implement predictive maintenance strategies, and coordinate distributed energy resources more effectively than with conventional infrastructure [13, 14, 15].

As shown in Figure 2.1, a smart grid typically implements a three-layer architecture comprising a power system layer, a communication layer, and an application layer that together enable the coordinated operation of the grid [12]. This hierarchical structure facilitates the integration of traditional power infrastructure with digital technologies while maintaining operational reliability through systematic deployment of monitoring and control capabilities [12].

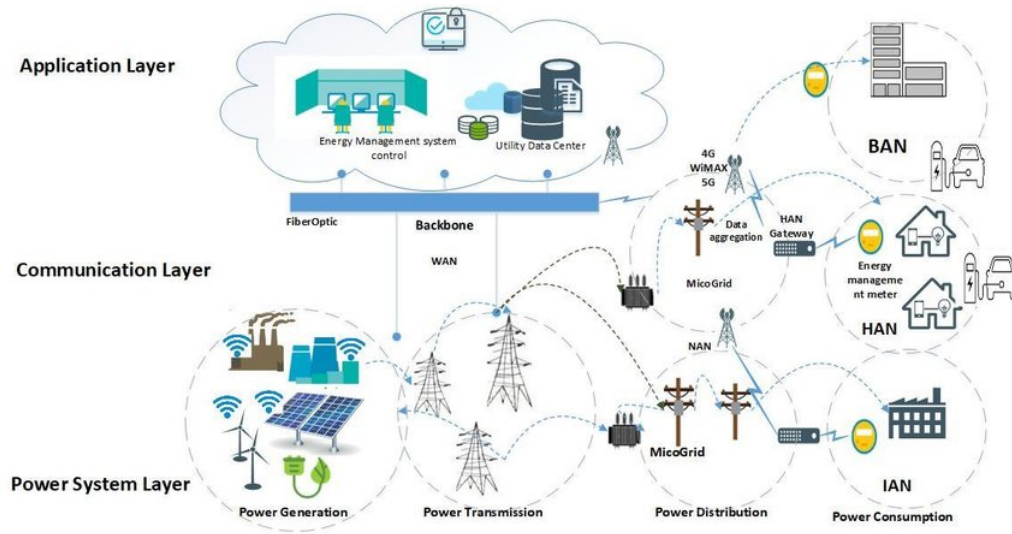


Figure 2.1: Smart Grid Three-Layer Architecture Model showing the integration of physical infrastructure, communication networks, and application services. Source: [16]

In Figure 2.1, the power system layer follows a hierarchical path from generation through transmission to distribution and finally consumption. At the generation and transmission stage, power plants feed electricity into the transmission network, which is spanned by a Wide Area Network (WAN) that interconnects substations and control centres over long distances via fibre-optic connections. The transmission network then branches into the distribution level, where the grid subdivides into multiple microgrids, which are semi-autonomous subsystems that can balance local generation and demand independently and, if necessary, island themselves from the main grid in case of faults in the upstream network [17, 18]. At this distribution level, a Neighbourhood Area Network (NAN) aggregates data from multiple residential or commercial premises within a district, linking individual consumer networks to the utility infrastructure. The microgrids deliver power to the consumption domain, which is organised into several network types visible in the figure: the Industrial Area Network (IAN) serving industrial facilities, the Home Area Network (HAN) covering individual households, and the Business Area Network (BAN) connecting commercial and utility stakeholders [12, 11].

Smart meters are deployed at the boundary between the distribution network and these consumption-side networks. Within the HAN, a HAN gateway collects meter data from one or more smart meters and transmits it upstream via Wi-Fi or wired Ethernet connections, bridging the in-home network to the utility communication infrastructure [11, 14].

Across the broader grid, these metering systems facilitate load forecasting, outage detection, and energy theft prevention while providing consumers with usage information that supports informed decision-making [14].

The communication layer implements diverse protocols, including 4G, WiMAX, and 5G, as well as power-line communication, wireless networks, and fibre-optic systems, to enable high-performance data exchange between grid components at every level of the hierarchy shown in Figure 2.1. The application layer hosts an energy management control system, such as SCADA, coupled to a utility data centre for data storage and analytics, to process the large volumes of grid data collected by the lower layers [12]. Energy management and SCADA systems coordinate operations across the grid, enabling the development of control algorithms that optimise system performance and stability under varying conditions [19]. These systems interact with market mechanisms and regulatory frameworks to enable economic dispatch of distributed resources through the BAN while maintaining reliability constraints [15].

The grid topology thus builds upon the three layers of Figure 2.1 by implementing hierarchical structures spanning transmission, distribution, microgrids, and the consumption-side area networks [17]. Through this coordinated approach to system architecture, smart grids achieve levels of efficiency and reliability that can accommodate increasing shares of distributed energy resources while maintaining grid stability [20].

2.2 SCADA SYSTEMS

SCADA systems are a combination of hardware and software components used to monitor and control technical processes in domains such as power plants, water treatment facilities, and power grids [21]. In the context of smart grids, a SCADA system collects data from distributed sensors, processes it centrally, and displays it on a supervisory interface, enabling operators to monitor system conditions, respond to events, and coordinate control operations [22].

As illustrated in Figure 2.2, the SCADA architecture is organised into five hierarchical layers spanning from the physical process to the central monitoring station [23].

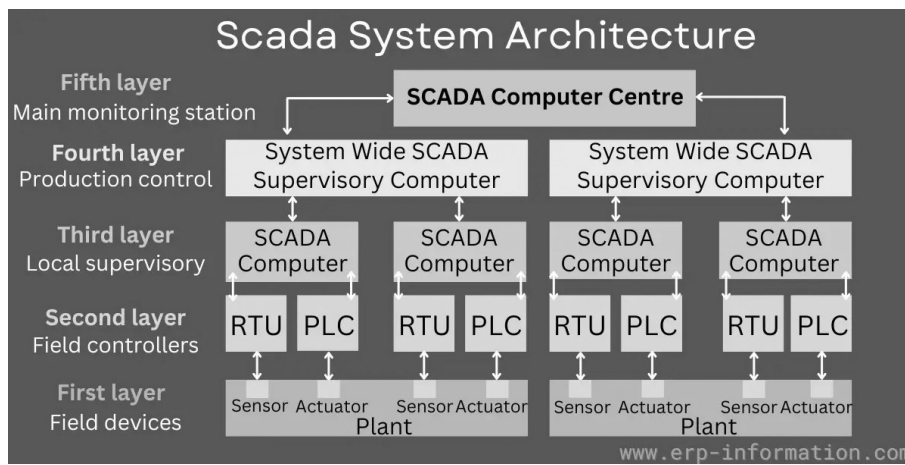


Figure 2.2: Typical SCADA system architecture organised in five hierarchical layers, from field devices at the bottom to the central monitoring station at the top [23].

At the lowest layer (*Field Devices*), sensors and actuators interface directly with the physical process. Sensors measure quantities such as voltage, current, temperature, or pressure, while actuators execute switching and control actions on the equipment [21].

The second layer (*Field Controllers*) hosts Remote Terminal Units (RTUs) and Programmable Logic Controllers (PLCs). RTUs collect readings from the sensors below and convert them into digital data that can be transmitted upstream, whereas PLCs are primarily responsible for local, automated control of actuators and process equipment [21, 22].

At the third layer (*Local Supervisory*), local SCADA computers aggregate data from multiple field controllers within a site or substation. These systems provide a first level of data processing and visualisation, allowing local personnel to supervise and intervene in the process without relying on the central station [22].

The fourth layer (*Production Control*) contains system-wide SCADA supervisory computers that consolidate information from several local supervisory nodes. At this level, cross-site data is correlated and higher-level control decisions are coordinated. The Master Terminal Unit (RTU) typically resides at this layer, aggregating measurements, issuing control commands, and storing operational data for subsequent analysis [21, 22].

The uppermost layer (*Monitoring Station*) houses the SCADA computer centre, where operators interact with the system through the HMI. The HMI translates raw process data into graphical visualisations such as trend diagrams, alarm lists, and control screens, thereby supporting Situational Awareness (SA) and informed decision-making [22]. A dedicated *historian* at this level continuously records process data, events, and alarms in a centralised database, enabling trend analysis, auditing, and reporting over extended time horizons [22, 21].

All five layers are interconnected by a *communication network* that carries data between sensors, field controllers, supervisory computers, and the monitoring station. This network may rely on wired media such as optical fibre or copper, wireless technologies such as GPRS or Wi-Fi, or a combination of both, using industrial communication protocols such as IEC 60870-5-104 or IEC 61850. Because every component depends on timely and accurate data exchange, the reliability and security of this network are fundamental to the overall performance and safety of the SCADA system [24, 21].

2.3 SITUATIONAL AWARENESS

Situational awareness is a concept that originated in military aviation research and has since been adopted across a wide range of safety-critical domains, including air traffic control, nuclear power plant operations, and process control [25]. In her seminal work, Endsley defines SA as “the perception of the elements in the environment within a volume of time and space, the comprehension of their meaning, and the projection of their status in the near future”[25]. This definition emphasises that SA is not merely the passive reception of data but an active cognitive process in which an operator integrates disparate information sources into a coherent mental model of the current and anticipated system state.

Endsley distinguishes SA from the closely related but distinct concepts of workload and decision-making. High workload does not necessarily imply poor SA, nor does low workload guarantee adequate SA; rather, SA represents the operator’s internal model of the environment that subsequently informs the decision-making process[25]. This distinction is central to the design of operator support systems: while reducing workload is desirable, the primary objective must be to ensure the operator maintains an accurate, up-to-date understanding of the system state.

Endsley’s Three-Level Model

Endsley’s model decomposes SA into three hierarchical levels, each building upon the preceding one[25]. The first and most fundamental level, Level 1 (*Perception*), involves detecting and recognizing relevant elements in the environment. In an operational context, this includes identifying current system states, active alarms, sensor readings, and the status of individual components. Failures at this level, for example, missing a critical alarm or overlooking an abnormal reading, propagate upward and compromise all higher levels of awareness.

At Level 2 (*Comprehension*), the operator synthesises the perceived elements into a coherent understanding of the situation. This entails recognising patterns, assessing the significance of observed values in relation to operational thresholds, and integrating information from multiple sources into a unified picture. For example, an operator who perceives both a rising transformer temperature and an increasing load on the associated feeder comprehends that the transformer may be approaching its thermal limit.

Level 3 (*Projection*) represents the highest level of SA and involves the ability to anticipate the future state of the system based on the current understanding. Projection enables proactive decision-making: rather than merely reacting to events as they occur, the operator can foresee potential problems and initiate preventive measures.

Endsley notes that Level 3 SA is the most cognitively demanding and therefore the first to degrade under high workload or stress conditions [25].

Together, the three levels form a processing pipeline: perception feeds comprehension, which in turn enables projection. Endsley's model further identifies several factors that affect the quality of SA, including the design of the information display, the complexity of the task environment, operator experience, and the degree of automation [25]. Automation, in particular, presents a double-edged effect: while it can reduce workload and support data fusion, poorly designed automation may cause operators to become complacent and lose awareness of the underlying system state, a phenomenon known as *out-of-the-loop* unfamiliarity[25].

The model has become the dominant theoretical framework for SA research, and standardised measurement instruments such as Situation Awareness Global Assessment Technique (SAGAT) and NASA Task Load Index (NASA-TLX) have been developed to assess operator SA and workload in controlled experiments [25].

Situational Awareness in Power Systems and Network Operations

In the context of power systems, SA is recognised as a principal determinant of effective grid operation. Grid operators must continuously monitor voltages, currents, power flows, switching states, and alarm streams across geographically distributed substations, often under strict time constraints [26]. Guttromson et al. identify SA as a critical human factor in grid operations and argue that the increasing complexity of modern grids demands decision-support tools that go beyond traditional alarm lists[26].

The transition from conventional distribution grids to smart grids amplifies these challenges. Stevens-Adams et al. observe that the ongoing modernisation of distribution utilities, including the deployment of smart meters, distributed generation, and automated switching equipment, fundamentally changes the flow of information available to control room operators. Their study of distribution control room operators in Vermont found that SA is a major contributor to successful operations, yet the impact of growing automation on operator SA remains poorly understood [27].

Modern SCADA systems support operator SA through advanced HMI interfaces that visualise system states using Single Line Diagram (SLD)s, trend graphs, and alarm notifications[28]. These interfaces reduce cognitive load by presenting data hierarchically and using colour coding to encode severity[29]. However, the sheer volume of data generated by smart grid infrastructure risks overwhelming operators, particularly during cascading fault scenarios in which hundreds of alarms may arrive within seconds [22].

Mapping Endsley’s three-level model to the power system domain illustrates the specific SA requirements of grid operators. At Level 1, the operator must perceive the current grid state: active alarms, voltage magnitudes, line loadings, and switching positions. At Level 2, the operator must comprehend what these readings mean in combination, for instance, recognising that a set of low-voltage alarms on adjacent feeders indicates a fault on a shared upstream bus. At Level 3, the operator must project the likely evolution of the situation, such as anticipating thermal overloads on alternative paths if load is rerouted [26]. A failure at any level can lead to delayed or incorrect switching decisions, potentially escalating a minor disturbance into a widespread outage.

These considerations motivate the design of operator support systems that explicitly target all three levels of SA: providing comprehensive and timely data displays (Level 1), contextualising events through rule-based evaluation and alarm correlation (Level 2), and offering trend visualisation and predictive indicators (Level 3).

Several existing systems address parts of this challenge. Edge Zero’s commercial platform performs predictive fault monitoring at over 300 000 measurement points in 1-minute intervals [30]. Adrah et al. describe an experimental energy management system that combines simulation, notifications, a rule engine, and SA metrics, yet their system lacks a dedicated operator HMI and is not integrated with smart grid infrastructure [31]. Salah et al. propose a model-driven approach to SCADA security that includes notifications, an HMI, SA components, anomaly detection, and a rule engine; however, the approach is not backed by simulation and does not address smart grid integration [32]. These works demonstrate that SA and anomaly detection are recognised as critical capabilities in the power systems domain, yet each solution addresses only a subset of the required building blocks, and none unifies them within a single operator support system.

2.4 HMI DESIGN AND ALARM CONCEPTS

HMIs serve as the primary interaction point between operators and SCADA systems, translating raw process data into comprehensible visualisations [21]. Effective HMI design is essential for maintaining operator SA and enabling rapid responses to system events [29]. Key design principles include clear visual hierarchies that organise information so as to prioritise critical data [28], consistent navigation patterns that ensure intuitive access to system controls and data views [33], and structured alarm management strategies that use colour coding and symbology to highlight anomalies and reduce cognitive load [29].

Modern HMI designs frequently incorporate SLDs to represent system topology and state. As illustrated in Figure 2.3, a SLD uses two-dimensional geometric elements such as rectangles and lines to depict individual components, while colour changes encode their operational status [28]. This topological representation enables operators to understand the spatial distribution of components and their conditions at a glance, and to coordinate field activities based on the current system state — for example, by toggling switch positions or adjusting setpoints [29]. Colour-based state visualisation helps operators notice changes more quickly, which is particularly important during high-stress scenarios in which multiple alarms may occur simultaneously [28, 29].

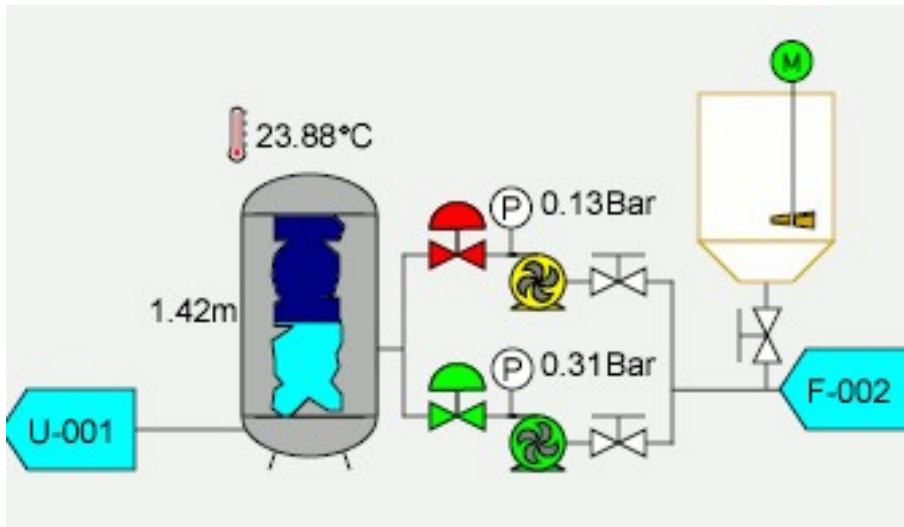


Figure 2.3: Illustration of a SCADA HMI screen showing a SLD with component labels and associated values [34].

The integration of real-time telemetry and analytics further enhances HMI functionality by providing predictive insights that support proactive decision-making. However, challenges remain in designing HMIs that balance informational completeness with usability, particularly as the volume of data generated by smart grid infrastructure continues to grow.

Existing research on SCADA interfaces addresses parts of this challenge. The Smart Grid Interface Module by EFEN implements IEC 60870-5-104 and IEC 61850 for distributed metering but does not provide higher-level operator support, such as notifications, rule evaluation, or SA enhancement [35]. Mahmood et al. present SCADASim, a simulation-based SCADA testbed with an HMI and notification capabilities, yet it does not include a rule engine, anomaly detection, SA mechanisms, or smart grid integration [36]. Pranggono and Arabo propose a SCADA security testbed that brings together simulation, an HMI, notifications, and SA features, but it omits anomaly detection, a rule engine, and smart grid coupling [37]. Engelmann et al. design a platform that focuses explicitly on the interplay between an HMI, notifications, and operator SA; their work is the closest to the present thesis in its operator-centric perspective, yet it does not include a simulation back-end, a rule engine, anomaly detection, or smart grid integration [38]. These contributions illustrate that SCADA interface research has largely focused on data acquisition or security, without embedding a comprehensive cognitive support pipeline for the operator.

2.5 Co-Simulation in CPES

A *co-simulation* is a simulation technique in which two or more simulators, each potentially based on different modelling paradigms, numerical solvers, and time-advancement schemes, are coupled such that they exchange data at defined synchronisation points and collectively reproduce the behaviour of an overall system [39]. In contrast to a monolithic simulation, where a single solver advances a single global system state, co-simulation treats each participating simulator as a *black box*: only its inputs and outputs are exposed to other participants, while its internal state, solver details, and implementation choices remain opaque. This separation is a major reason co-simulation is attractive for multi-domain systems, since established and validated tools can be reused in their native form rather than being reimplemented in a common environment [40].

The coordination of the participating simulators is handled by a central component commonly referred to as the *orchestrator*. Its responsibilities include initialising each simulator, scheduling simulation steps, and routing exchanged data at each synchronisation point [39]. Because coupled simulators may operate with different step sizes and even different time paradigms — for example, fixed-step continuous or discrete-time components versus discrete-event components — the orchestrator must reconcile these heterogeneous notions of time into a consistent global progression. A central practical concern is the trade-off between accuracy and runtime overhead: more frequent synchronisation reduces coupling error caused by stale exchanged values but increases communication and coordination costs [39]. Consequently, the chosen synchronisation strategy becomes part of the experimental design and must be reported when interpreting results.

Co-simulation is particularly well suited to Cyber-Physical Energy System (CPES), where electrical grids, communication networks, control strategies, and human-operated interfaces interact tightly. Analysing these layers in isolation can yield misleading conclusions, since emergent phenomena — such as protection actions affected by communication delay, or cascading failures triggered by uncoordinated control responses — only become visible once the layers are coupled [40]. Tools such as *mosaik* provide the orchestration infrastructure needed to combine heterogeneous simulators and enable reproducible experimentation: scenarios can be planned, executed, and analysed under controlled conditions, making co-simulation a suitable foundation for the evaluation of complex, multi-component smart grid systems [41].

The mosaik framework has been extended in various directions. Astero et al. integrate power electronics simulators through the Functional Mock-up Interface (FMI) standard, enabling the representation of complete cyber-physical energy systems in a single co-simulation run [42]. The National Renewable Energy Laboratory (NREL) project “Situational Awareness of Grid Anomalies” (SAGA) employs the HELICS framework to co-simulate transmission- and distribution-level phenomena at scale [43]. Zhang et al. propose an asynchronous event-driven co-simulation architecture with notifications and a rule engine, but their work lacks a graphical operator interface and does not target SA or smart grid integration [44]. Förster et al. present an OMNeT++-based network simulation with notifications and rule-based processing, yet similarly omit an HMI and do not address SA [45]. Stević et al. demonstrate a global real-time simulation testbed combining co-simulation with an HMI, anomaly detection, and direct smart grid integration; however, their platform provides neither a notification system nor a rule engine and does not explicitly target operator SA [46]. Across these contributions, co-simulation is primarily treated as an evaluation or integration tool — none of the reviewed frameworks couples a co-simulation back-end with a full notification and suggestion pipeline to enhance operator SA.

2.6 RELEVANT TECHNOLOGIES IN COMPUTER SCIENCE

This section presents the programming languages, data formats, algorithms, technologies, and architectural patterns used in the system’s implementation.

Python

Python is a high-level, interpreted, general-purpose programming language designed with an emphasis on readability, productivity, and a clear, expressive syntax [47]. It supports multiple programming paradigms, including procedural, object-oriented, and functional styles, and is widely used in domains ranging from web development and automation to scientific computing, data analysis, and machine learning [47]. The language is developed under an open-source model and is maintained by the Python Software Foundation (PSF), whose mission is to promote, protect, and advance the Python programming language and support its global community [47].

HDF5

The Hierarchical Data Format version 5 (HDF5) is a high-performance data management suite which combines a binary file format with a portable software library to store, process, and exchange large and heterogeneous scientific datasets [48, 49]. It has become a de facto standard for data management in many scientific and engineering domains, with widespread adoption across academic research, government laboratories, and industrial sectors such as aerospace, automotive, medical imaging, and financial services [48, 49]. The reference implementation of HDF5 is developed and maintained by The HDF Group and is available as an open-source C library with bindings for multiple programming languages [48, 49].

At the data model level, HDF5 organizes information in a hierarchical structure of groups and datasets, analogous to a file system, and supports n -dimensional datasets whose elements may themselves be compound or variable-length types [49]. A central characteristic is that HDF5 files are self-describing: both data and metadata (such as types, shapes, attributes, and semantic annotations) are stored together in a single portable binary container [49]. This design facilitates long-term preservation, reproducibility, and the exchange of complex data products without external schema descriptions, while avoiding vendor lock-in through its open specification [49].

From a systems perspective, HDF5 is engineered for high-performance I/O on platforms ranging from laptops to massively parallel supercomputers [49]. The library exposes a rich set of performance features, including chunked storage, compression filters, dataset caching, and parallel I/O through MPI-IO, enabling applications to optimise both access

latency and storage footprint for large-scale workloads [49]. Current releases are primarily implemented in C, with language bindings for C++, Fortran, and Java [48].

In the context of power system and cyber-physical energy system co-simulation, a single HDF5 file can encapsulate both static entity–relationship information (e.g. buses, lines, transformers, and topology encoded as groups, attributes, and tabular datasets) and high-frequency time series metering data (e.g. voltages, currents, and powers stored as multi-dimensional datasets). This layout exploits the hierarchical, self-describing HDF5 data model to co-locate grid models and their trajectories in one portable container, which is particularly suitable for efficient storage, replay, and analysis of large co-simulation traces and grid measurements [48, 49, 50].

Event Bus

An *event bus* is a software architectural pattern that decouples system components through indirect, message-based communication. Instead of calling each other directly, components *publish* typed events onto a shared bus, and the bus *dispatches* each event to every component that has previously declared interest in that event type—the *subscribers* [51]. The pattern is a specialisation of the Observer pattern described by Gamma et al. [52], extended with a centralised mediator that removes the need for publishers to maintain references to their subscribers and vice versa.

This indirection yields three principal benefits. First, *loose coupling*: publishers and subscribers share only the event type definition, not concrete class interfaces, which simplifies independent development and testing. Second, *extensibility*: new subscribers can be added at runtime without modifying existing publishers. Third, *composability*: a single event can trigger processing in arbitrarily many handlers, enabling cross-cutting concerns such as logging, telemetry, or auditing to observe the same event stream that drives the core business logic [53].

Eugster et al. [53] identifies three fundamental dimensions along which publish/subscribe systems vary: the *subscription model* (topic-based, content-based, or type-based), the *delivery guarantee* (at-most-once, at-least-once, or exactly-once), and the *ordering guarantee* (FIFO, causal, or total order). The event bus described in this work adopts a *topic-based* subscription model where each event carries a string-valued type that serves as the routing key, with *at-most-once* delivery and *per-topic FIFO* ordering, which is the strongest ordering that can be provided without cross-topic coordination overhead.

Ring Buffers

A *ring buffer* (also called a *circular buffer* or *circular queue*) is a fixed-size, contiguous data structure that logically wraps around: once the write position reaches the end of the underlying array, it continues at position zero, thereby reusing storage without any memory allocation or deallocation at runtime [54]. Two indices, conventionally called *head* (write position) and *tail* (read position), track where the next element is inserted and consumed, respectively. An element is *enqueued* by writing it at the head index and advancing that index by one; it is *dequeued* by reading from the tail index and advancing accordingly. The buffer is considered *full* when the number of stored elements equals the fixed capacity, and *empty* when head and tail coincide [54].

Because a ring buffer never moves existing elements and operates through simple index arithmetic, it provides $O(1)$ enqueue and dequeue operations. This constant-time behaviour makes ring buffers attractive wherever bounded, high-throughput queuing is required, from operating-system IO subsystems and audio processing pipelines to, as in the present work, event buses in real-time simulation software.

Power-of-Two Sizing and Bitwise Masking

A common optimization rounds the buffer capacity to the next power of two. When the capacity C satisfies $C = 2^k$ for some $k \in \mathbb{N}$, the modulo operation used to wrap indices can be replaced by a bitwise AND with the mask $C - 1$ [55]:

$$\text{index}_{\text{physical}} = \text{index}_{\text{logical}} \ \& \ (C - 1) \quad (2.1)$$

This substitution avoids the comparatively expensive integer division that a general modulo entails, yielding a measurable throughput improvement on modern processors where branch-free arithmetic dominates the critical path [56].

Lock-Free and Synchronized Variants

In a single-threaded context, the ring buffer requires no synchronisation. Once multiple threads are involved, however, the question arises which operations must be protected and at what granularity. Lamport showed that a single-producer, Single Producer Single Consumer (SPSC) bounded buffer can be implemented without mutual exclusion: because the producer only modifies the head index and the consumer only modifies the tail index, no two threads ever write to the same memory location, and ordinary load/store ordering suffices on most architectures [57].

When more than one thread must perform the same role, multiple producers, multiple consumers, or both, some form of synchronization becomes necessary to prevent data races on the shared index. The design space ranges from fully lock-free algorithms based on atomic compare-and-swap (CAS) instructions [58] to coarser-grained approaches that protect critical sections with mutual-exclusion locks [59]. The choice depends on the expected contention pattern: lock-free algorithms scale better under high contention, whereas a simple mutex can outperform CAS loops when contention is rare, because the latter incur retry overhead even in the uncontended case [60].

Core Ring-Buffer Operations

The fundamental operations shared by all four variants are the buffer initialisation, the `PUSH` (enqueue) operation, and the `POP` (dequeue) operation. Algorithm 1 presents these procedures in pseudocode.

The algorithm proceeds as follows. In `INIT` (lines 1–7), the requested capacity is rounded up to the next power of two (line 2) so that a bitmask can replace the modulo operation in all subsequent index computations (line 3). The backing array is then allocated with C slots initialised to `NIL` (line 4), and both head and tail pointers start at zero (lines 5–6).

`PUSH` (lines 8–14) first checks whether the buffer is at capacity (line 9); if so, the event is rejected, and the caller receives `FALSE`. Otherwise, the event is written at the physical index obtained by masking the head pointer (line 11), this is precisely the bitwise-AND optimisation from Equation (2.1). The head pointer is then incremented (line 12); because it is allowed to grow without bound while only the masked value is used for indexing, no explicit wrap-around branch is needed.

`POP` (lines 15–22) is the mirror image: if the buffer is empty, the function returns `NIL` (line 16). Otherwise, the event at the tail position is retrieved (line 18), the slot is cleared to assist Python’s garbage collector (line 19), and the tail pointer advances (line 20).

Batch Dequeue

For throughput-critical paths, the buffer also provides a batch-dequeue operation, shown in Algorithm 2. Rather than incurring function-call overhead per event, `POPBATCH` removes up to n events in a single invocation.

Line 2 caps the dequeue count at the number of events actually present, avoiding underflow. The loop (lines 7–12) advances the tail pointer one element at a time, collecting events into a result list while clearing each slot. Notably, the size counter is decremented only once after the loop (line 13) rather than inside it, which reduces the number of writes to the shared variable, a minor but measurable optimization when k is large.

Algorithm 1 Ring-buffer initialisation, push, and pop (SPSC variant).

```

1: procedure INIT(capacity)
2:    $C \leftarrow 2^{\lceil \log_2 \text{capacity} \rceil}$                                 ▷ Round up to next power of two
3:    $\text{mask} \leftarrow C - 1$                                           ▷ Pre-compute bitmask
4:    $\text{buf}[0 \dots C - 1] \leftarrow \text{NIL}$                              ▷ Allocate fixed-size array
5:    $\text{head} \leftarrow 0$                                               ▷ Write position (monotonically increasing)
6:    $\text{tail} \leftarrow 0$                                              ▷ Read position (monotonically increasing)
7:    $\text{size} \leftarrow 0$                                              ▷ Current element count
8: end procedure

9: function PUSH(event)
10:  if  $\text{size} \geq C$  then
11:    return FALSE                                                ▷ Buffer full , reject
12:  end if
13:   $\text{buf}[\text{head} \ \& \ \text{mask}] \leftarrow \text{event}$                     ▷ Write at masked index
14:   $\text{head} \leftarrow \text{head} + 1$                                        ▷ Advance write cursor
15:   $\text{size} \leftarrow \text{size} + 1$ 
16:  return TRUE
17: end function

18: function POP
19:  if  $\text{size} = 0$  then
20:    return NIL                                                  ▷ Buffer empty
21:  end if
22:   $e \leftarrow \text{buf}[\text{tail} \ \& \ \text{mask}]$                         ▷ Read at masked index
23:   $\text{buf}[\text{tail} \ \& \ \text{mask}] \leftarrow \text{NIL}$                     ▷ Clear slot (aid garbage collection)
24:   $\text{tail} \leftarrow \text{tail} + 1$                                        ▷ Advance read cursor
25:   $\text{size} \leftarrow \text{size} - 1$ 
26:  return  $e$ 
27: end function

```

Algorithm 2 Batch dequeue from a ring buffer.

```

1: function POPBATCH( $n$ )
2:    $k \leftarrow \min(n, size)$  ▷ Clamp to available events
3:   if  $k = 0$  then
4:     return  $\langle \rangle$  ▷ Empty list
5:   end if
6:    $result \leftarrow \langle \rangle$ 
7:   for  $i \leftarrow 1$  to  $k$  do
8:      $idx \leftarrow tail \ \& \ mask$ 
9:     APPEND( $result$ ,  $buf[idx]$ )
10:     $buf[idx] \leftarrow NIL$ 
11:     $tail \leftarrow tail + 1$ 
12:   end for
13:    $size \leftarrow size - k$  ▷ Single decrement after loop
14:   return  $result$ 
15: end function

```

Timeseries Databases

Time-series databases are specialized data management systems optimized for storing, ingesting, and querying data indexed primarily by time, such as sensor readings, metrics, and event logs [61]. In contrast to traditional relational workloads, time-series workloads are characterized by high write rates, largely append-only patterns, and queries that frequently aggregate or filter over time ranges (for example, “last 24 hours” or “last 30 days”) [61]. In many cyber-physical and energy systems, meter data is produced at fixed intervals (e.g., every 15 minutes) and therefore naturally forms long, ordered sequences of measurements that are well-suited to time-series-oriented storage and access patterns.

TimescaleDB represents a state-of-the-art approach to time-series data management by implementing time-series capabilities as an extension on top of PostgreSQL rather than as a separate standalone database [61]. It is described as an open-source PostgreSQL extension designed for real-time analytics on time-series data, maintaining full SQL compatibility while adding specialized storage and query optimizations [61]. This design allows applications to retain the mature transactional and relational features of PostgreSQL (ACID semantics, joins, indices, JSON support) while benefiting from higher performance on time-series workloads [61].

Architecturally, TimescaleDB introduces abstractions such as **hypertables**, which automatically partition underlying data by time and optional key dimensions to improve write throughput and query performance [62]. Additional features include continuous aggregates for precomputing rollups over time windows, compression of historical data, and tiered stor-

age strategies that keep recent “hot” data on fast storage while moving older “cold” data to cheaper media [61, 62]. These mechanisms are particularly relevant for infrastructures that collect high volumes of telemetry over long periods, such as smart grid metering and SCADA-like monitoring systems, where raw measurement streams and derived aggregates must both remain queryable at scale [62].

Grafana

Grafana is an open-source visualization and analytics platform for exploring, querying, and presenting time-series and event data from a wide range of data sources [63]. Originating as a dashboarding solution for metrics backends such as Prometheus and Graphite, Grafana has evolved into a central component of modern observability stacks, capable of integrating metrics, logs, and traces into unified views of system behaviour [63]. The platform is developed and maintained by Grafana Labs, which provides both an open-source edition and a managed Grafana Cloud offering that extends the core capabilities with hosted storage, alerting, incident response tooling, and AI-assisted analysis [63].

At its core, Grafana provides a flexible query and visualization layer that sits on top of existing telemetry stores rather than replacing them [63]. It supports numerous data sources, including time series databases, log backends, and distributed tracing systems, and allows users to define dashboards composed of panels that each issue queries and render results using rich visualizations such as time series graphs, heatmaps, tables, and geomaps [63]. This decoupled approach enables organizations to adopt Grafana as a single, consistent interface for heterogeneous backends, reducing the need for bespoke tooling per system while retaining the ability to leverage backend-specific query languages [63].

3

Requirements & Evaluation Design

The preceding chapter established the theoretical and technical foundations for the proposed system. Before proceeding to the concrete design and implementation, it is necessary to specify *what* the system must accomplish and *how* its success can be measured. This chapter, therefore, serves a dual purpose. Section 3.1 formulates the functional and non-functional requirements that govern the system’s scope, translating the research questions and the gaps identified in the literature into concrete engineering constraints. Section 3.2 then develops the evaluation methodology for assessing the resulting implementation, defining experimental factors, quantitative benchmarks, and proxy-based situational awareness metrics that together form the basis for the empirical analysis in the later chapters.

3.1 SYSTEM REQUIREMENTS

The Smart-Grid-Ally (Sally) system is conceived as a SCADA extension for extracting critical information from timeseries smart grid data and presenting it to operators through structured notifications and suggestions in grid operation and training scenarios. The following requirements are derived from the overall project goals, the research questions, and the concrete architecture and implementation outlined in the later chapters.

The following requirements were derived through a requirements analysis, taking my supervisor’s requirements, requirements identified through the literature review, and the Foundations into account to define the requirements of the proposed system.

3.1.1 Functional requirements

FR1: Data Ingestion and Scenario Replay. Sally shall ingest time series measurements and grid state information from co-simulation outputs (HDF5 exports) and, optionally, from a TimescaleDB instance that mirrors real-world time series and entity-relational data. The ingestion must support deterministic replay of scenarios with consistent timestep semantics so that identical input trajectories can be used under varying configurations.

FR2: Rule-Based Event Detection. The system shall evaluate user-defined rules over incoming grid data to detect conditions of interest, such as voltage limit violations, frequency deviations, or abnormal power flows. Rules must support a rich operator set (e.g., greater/less than, ranges, equality with tolerance, string pattern matching) and allow logical chaining to form composite conditions.

FR3: Notification and Suggestion Generation. Upon rule triggering, SALLY shall generate notifications that summarize the detected condition, identify affected entities in the grid model, and optionally attach suggestions for possible operator actions (e.g. setpoint adjustments or reconfiguration steps). Notifications must be structured so that their content can be rendered consistently across different GUI components without encoding interface-specific assumptions.

FR4: Multi-Interface GUI Integration. The system shall present notifications and rule status information in at least three GUI contexts:

- a SCADA-style display with simulation controls, a rule status overview, a Single-Line Diagram (SLD) visualization with color-coded component states and overlays for rule-triggered events.
- a Rule Manager GUI for creating, editing, and managing rules, and
- a Single-Line Diagram (SLD) visualization with color-coded component states and overlays for rule-triggered events.

All interfaces are implemented using Python's `tkinter` and related libraries and must be driven by the same backend state.

FR5: SCADA Orchestration and Event Distribution. A SCADA Orchestration Service shall coordinate simulation stepping, data retrieval from HDF5 or TimescaleDB, rule evaluation, and state updates, publishing domain events to a dedicated event bus. The event bus provides decoupled communication between backend services and GUIs via typed domain events such as *GridDataEvent*, *RuleTriggeredEvent*, and *SimulationStepEvent*.

FR6: Telemetry and Observability. SALLY shall emit telemetry data (metrics, traces, logs) for all major components using Open Telemetry (OTEL), exporting them to a Grafana-based stack that includes Prometheus, Tempo, and Loki. Pre-configured dashboards shall provide an overview of simulation progress, event bus activity, rule evaluation, and SCADA display behaviour using standardized metric names with a common `sally_` prefix.

FR7: Experiment Logging and Export. The system shall log experiment runs, including scenario identifiers, rule set versions, notification configurations, and key performance metrics, in a format suitable for statistical analysis (e.g. TimescaleDB tables with CSV export). Log entries must be linkable via correlation identifiers to telemetry traces and measurement time steps.

3.1.2 Non-functional requirements

NFR1: Latency and Throughput. End-to-end latency from data ingestion (per simulation timestep or measurement batch) to notification availability in the GUIs should remain below a sub-second threshold under nominal load. The event bus is designed for high-throughput scenarios with performance targets on the order of millions of events per second, using ring buffers and batch processing strategies to reduce per-event overhead.

NFR2: Scalability and Concurrency. The architecture shall sustain high event rates and large rule sets by selecting appropriate buffer types (SPSC, Single Producer Multiple Consumer (SPMC), Multiple Producer Single Consumer (MPSC), Multiple Producer Multiple Consumer (MPMC)) for the event bus and allowing multiple worker threads per event type. Scalability requirements include the ability to adjust batch sizes, worker counts, and buffer capacities without changing business logic.

NFR3: Reliability and Fault Tolerance. The system shall degrade gracefully under overload, with dropped or delayed events counted and exposed through telemetry metrics so that overload conditions are diagnosable. Component failures (e.g. in data providers or GUIs) shall not corrupt core domain state, and recovery or restart procedures must be supported via clear separation between core, domain, infrastructure, and presentation layers.

NFR4: Maintainability and Testability. The backend follows a layered architecture (core, domain, infrastructure, application, presentation) with dependency injection to allow isolated testing and replacement of data providers, rule engines, or GUIs. All components shall be amenable to unit testing and integration testing without requiring a full deployment of the observability stack or external databases.

NFR5: Extensibility and Integration. The system shall allow integration of additional anomaly detection algorithms, data sources, or GUIs via well-defined service interfaces and dependency injection configuration. Integration with external SCADA web frontends or monitoring systems (e.g. via MQTT or HTTP bridges) must be possible without modifications to the core event bus or domain model.

NFR6: Reproducibility and Configuration Management. Scenario definitions, rule sets, and orchestration parameters shall be configurable via YAML files and environment variables, and these configurations must be versionable and documented. Experiments must be reproducible when the same scenario, configuration, and software version are used.

3.2 EVALUATION DESIGN

The evaluation design focuses on validating the correctness, performance, and robustness of SALLY as a SCADA extension. It combines functional validation of notification and rule behaviour with quantitative measurements of throughput, latency, and resource usage, using the observability stack across different testing and telemetry approaches.

3.2.1 Evaluation Objectives

The evaluation addresses three main objectives

1. **Functional correctness:** Validate that rules are evaluated as specified, that notifications are generated for the correct conditions, and that Graphical User Interface (GUI)s reflect backend state consistently during simulation.
2. **Performance and scalability:** Quantify whether the event bus, orchestration service, and GUIs meet the targeted throughput and latency bounds for high-frequency event streams.
3. **Situational awareness support:** Evaluate, through automated proxy metrics mapped to Endsley's three-level SA model, whether the system provides operators with the information needed for perception, comprehension, and projection of grid states, without requiring human-subject experiments.

3.2.2 Experimental Factors and Scenarios

The technical evaluation is structured around replayable scenarios and synthetic load

- **Scenario complexity.** Multiple grid scenarios of different sizes and disturbance patterns are generated via mosaik and midas and exported as HDF5 files.
- **Rule configurations.** For each scenario, different rule sets are applied, varying the number of active rules, the complexity of conditions, and the presence of chained rules to study the impact on evaluation cost and notification volume.
- **Load profiles.** Synthetic load tests accelerate replay (shorter timestep intervals, increased event rates) and use dedicated load scripts to publish large numbers of events over defined durations to the event bus and telemetry endpoints.

These factors allow systematic variation of grid size, rule complexity, and event rate while keeping the co-simulation trajectories themselves fixed for comparability.

3.2.3 Proxy-Based Situational Awareness Evaluation

A central design goal of SALLY is to improve operator SA in SCADA environments. Evaluating SA classically requires human-subject studies using instruments such as SAGAT freezes, SART questionnaires, or NASA-TLX workload measurements [64]. Because the scope of this thesis precludes controlled user studies, the evaluation instead employs a set of *automated proxy metrics* that map onto the three levels of Endsley’s SA model and can be computed entirely from system telemetry, event bus traces, and co-simulation ground truth.

Level 1 – Perception Proxies

Level 1 SA concerns whether operators can perceive all relevant elements of the current situation. The following automated checks serve as proxies:

- **Display completeness ratio.** For each simulation timestep t , let $E_{\text{gt}}(t)$ be the set of entity–attribute pairs present in the HDF5 ground truth, and $E_{\text{gui}}(t)$ the set actually propagated to the GUI via domain events. The display completeness is defined as

$$C(t) = |E_{\text{gui}}(t) \cap E_{\text{gt}}(t)| / |E_{\text{gt}}(t)|.$$

A value below 1 indicates that the operator would lack visibility of some grid state elements. Because the numerator is an intersection with $E_{\text{gt}}(t)$, it can never exceed the denominator; hence $C(t) \in [0, 1]$ by construction, and any spurious or duplicated entries that the GUI might receive do not inflate the score. The metric operates on the raw entity–attribute pairs propagated through the event bus, not on derived or aggregated representations. Aggregated views such as the colour-coded SLD state, which maps a continuous measurement to a discrete severity level, are evaluated separately by the SLD state accuracy metric defined below, so that $C(t)$ remains a pure measure of data-level coverage.

- **SLD state accuracy.** For each entity and timestep, the expected visual severity (nominal, warning, critical) is derived from the ground-truth measurement against the configured rule thresholds. The SLD state accuracy is the fraction of entities whose color-coded state on the SLD matches this expected severity.

Level 2 – Comprehension Proxy

Level 2 concerns whether the presented information allows the operator to understand the meaning of the current situation.

The proxy for this level is **notification structural completeness**, which measures whether each emitted notification carries sufficient context for an operator to comprehend the detected condition without consulting secondary data sources.

For each notification n emitted via the event bus, let F_{req} be the set of required contextual fields, entity identifier, measurement attribute, current value, threshold value, severity level, and timestamp, and let $F_{\text{present}}(n) \subseteq F_{\text{req}}$ be the subset of those fields that are non-empty in the payload. The per-notification structural completeness is defined as

$$S(n) = \frac{|F_{\text{present}}(n)|}{|F_{\text{req}}|}$$

and the scenario-level score is the mean $\bar{S} = \frac{1}{N} \sum_{i=1}^N S(n_i)$ over all N notifications in the run. A score of $\bar{S} = 1.0$ confirms that every notification provides the full information an operator would need; a value below 1 pinpoints which field categories are missing. This metric is straightforward to implement as an automated assertion loop over the collected `RuleTriggeredEvent` payloads and requires no external ground-truth evaluator.

Level 3 – Projection proxy

Level 3 addresses whether the system enables the operator to anticipate near-future states. This is achieved using **Trend-data availability**, where for each threshold violation, verify that at least w preceding timesteps of the relevant measurement attribute were available in the data pipeline, where w is a configurable history window. This proxy assesses whether an operator could have observed an approaching violation from trend information prior to the alarm.

3.2.4 Metrics and measurement

The system's evaluation is based on a comprehensive set of metrics and testing strategies. Throughput metrics include the event publication rate on the event bus per second, and the rule trigger rate under varying rule configurations. Latency metrics are captured as histogram-based percentiles (P50, P95, P99) for per-event processing latency from publication to handler completion, using OTEL metrics, as well as the end-to-end delay from a simulation step request to the corresponding SCADA display and SLD update, derived from correlated traces and logs. Correctness indicators include consistency checks between HDF5 source data and GUI-displayed values for selected entities and timesteps, as well as automated assertions on rule evaluation outcomes for known test cases, ensuring that changes in the data or applied setpoints trigger the expected subset of rules.

4 | System Design Concept

With the requirements and evaluation strategy defined in the preceding chapter, the next step is to translate those constraints into a concrete system design. This chapter presents the conceptual blueprint of SAlly before introducing any implementation details. Every design decision is guided by the overarching goal of supporting operator SA at all three levels of Endsley’s model: the GUI concepts target perception (Level 1) through comprehensive grid visualisation and comprehension (Level 2) through structured notifications, while the architecture ensures that the underlying data pipeline is fast and reliable enough to enable projection (Level 3) from trend information. Section 4.1 describes the planned graphical interfaces through which operators interact with the system, outlining the layout and intended capabilities of the Rule Manager and the SCADA display. Section 4.2 then develops the overall architecture, showing how data ingestion, rule evaluation, event distribution, and visualization are organized into loosely coupled layers that satisfy the functional and non-functional requirements established in Chapter 3.

4.1 GUI INTEGRATION CONCEPT

The GUI integration concept covers the planning of the two main GUIs and gives a non-technical overview of their layout and planned capabilities.

4.1.1 Rule-Manager GUI

The Rule-Manager GUI enables operators to create, edit, and manage conditional rules that are evaluated against grid measurements at each simulation step or in real time using streaming data from a database. Rules can trigger alerts or programmable actions.

The Rule Manager window is divided into three main areas, as shown in Figure 4.1. At the top, a *Control Frame* provides file operations for saving and loading rule sets, discovery of available HDF5 scenario files, and synchronization controls that allow the Rule Manager to connect to the orchestrator and receive entity metadata from a TimescaleDB instance. Below the control bar, the window splits vertically into two panes. The left pane houses the *Rule Creator Frame*, a form-based interface for defining new rules or editing existing ones. Each rule is specified by selecting a grid entity and one of its measurement attributes, choosing a comparison operator (e.g. greater than, less than, range, or pattern match), setting a threshold value, and assigning an action type together with a severity level. Rules can be assigned to a named group and combined with logical operators (AND, OR) to

express composite conditions spanning multiple entities or attributes as a single chained rule. The right pane contains the *Rules View Frame*, a tabular overview of all currently defined rules with columns for the rule identifier, entity, variable, operator, threshold, action, and active status. Operators can edit rules inline, toggle individual rules on or off, or delete them directly from the table.

The Rule Manager is designed as a standalone application that can be used independently of a running simulation. Rule sets are persisted as portable configuration files, enabling rule libraries to be prepared and versioned offline for different grid scenarios. When a simulation or live session starts, the orchestrator loads the selected rule set and evaluates it at every timestep, allowing the same rules to be reused across replayable training runs and, in principle, during live operation with streaming data.

Rule Manager

Load HDF5
Save Rules
Load Rules
Sync Status

Rule Creator

Entity:

Variable:

Operator:

Threshold:

Action:

Group:

Logic Op:

Add Rule

Rules Table

ID	Entity	Variable	Op	Value	Action	Active

Figure 4.1: Rule Manager GUI Layout

4.1.2 SCADA Display

The SCADA-and Setpoint-GUI is the primary operational interface, providing real-time visualization of grid state through a SLD, simulation controls such as stepping and “playing” (continuous stepping with configurable timeout), as well as setpoint management and a rule status display to view triggered rules and their alerts.

The SCADA window uses a paned layout with a collapsible sidebar for “Controls” and a main display area for the SLD visualization, as shown in Figure 4.2.

The SLD renders the grid topology as a single-line diagram in which every component, including generators, loads, transformers, and lines, is colour-coded according to its current measurement values and the active rule thresholds. Components operating within nominal bounds appear in a neutral colour, those approaching a threshold transition to a warning state, and entities that have triggered a rule are highlighted in an alarm colour, giving the operator an immediate visual overview of the grid condition at any point in time.

The sidebar groups its controls into several panels. The *Simulation* panel displays the current timestep and provides buttons for single-stepping, continuous playback at a configurable speed, and pausing or resetting the replay. The *Setpoint Control* panel allows the operator to select an entity and submit a new setpoint value, which the orchestrator applies before the next evaluation cycle. All applied setpoints are listed in a dedicated *Applied Setpoints* panel, providing the operator with an audit trail of manual interventions. At the bottom of the sidebar, the *Rule Status* panel lists all rules that are currently triggered, together with their severity, affected entity, and the measurement value that caused the trigger. This panel is the primary entry point for notifications: when the orchestrator publishes a *RuleTriggeredEvent* on the event bus, the SCADA display consumes it and updates both the rule status list and the corresponding SLD overlay in the same display cycle, ensuring that the textual notification and the visual highlight remain consistent.

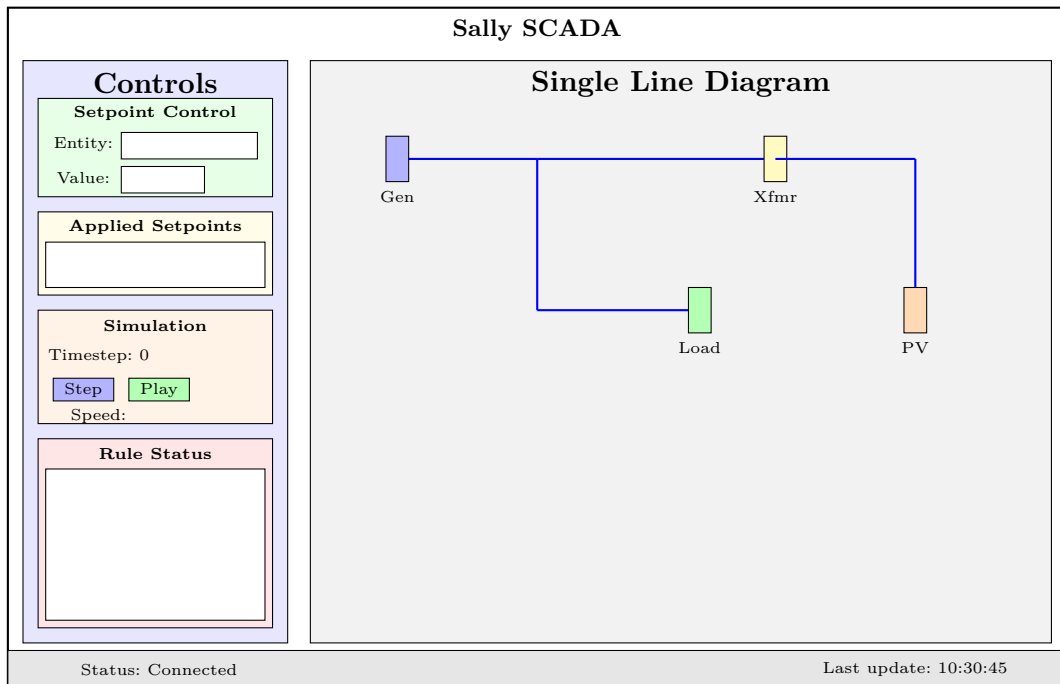


Figure 4.2: SCADA Display Layout

4.2 PLANNED ARCHITECTURE

Figure 4.3 provides a high-level view of the Sally architecture.

At the top, the *data source layer* provides the raw grid measurements on which all downstream processing operates. Two providers are supported: an HDF5 file reader for deterministic replay of mosaik-generated co-simulation exports, and a TimescaleDB connector for live or near-live time-series ingestion. In simulation mode with HDF5, the entity–relation data is stored in the file, unlike in TimescaleDB, where it must be evaluated at runtime. Both providers expose the same data-provider interface, so that the remainder of the pipeline is agnostic to the data origin.

The data source feeds into the *simulation and orchestration layer*, where the Orchestrator coordinates each processing cycle. On every timestep, the Orchestrator retrieves the current entity measurements, forwards them to the Rule Manager for threshold and chain evaluation, collects the resulting rule triggers, and publishes two typed domain events—**GridDataEvent** and **RuleTriggeredEvent**—onto the Event Bus. The Event Bus, built on the ring-buffer variants described in Section 2.6, decouples publishers from their subscribers and prevents high-frequency event streams from blocking any single consumer.

Downstream of the Event Bus, the *presentation and observability layer* contains three principal consumers. The SCADA display with its embedded SLD and Setpoint interface subscribes to both event types so that the operator sees updated grid states and rule-triggered highlights in the same display cycle. The Rule Manager GUI, when synchronised, also subscribes to rule events to provide colour-coded feedback for triggered conditions. In parallel, the OTEL instrumentation layer exports metrics, traces, and structured logs to the Grafana stack (Prometheus, Tempo, Loki), providing an independent observability view of system behaviour that is primarily valuable for developers, trainers reviewing session telemetry, and future integration testing.

This separation into data source, orchestration, and presentation layers directly satisfies the modularity and extensibility requirements (NFR4, NFR5) established in Chapter 3: each layer can be developed, tested, and replaced independently, and new consumers or data sources can be added without modifying the core event pipeline.

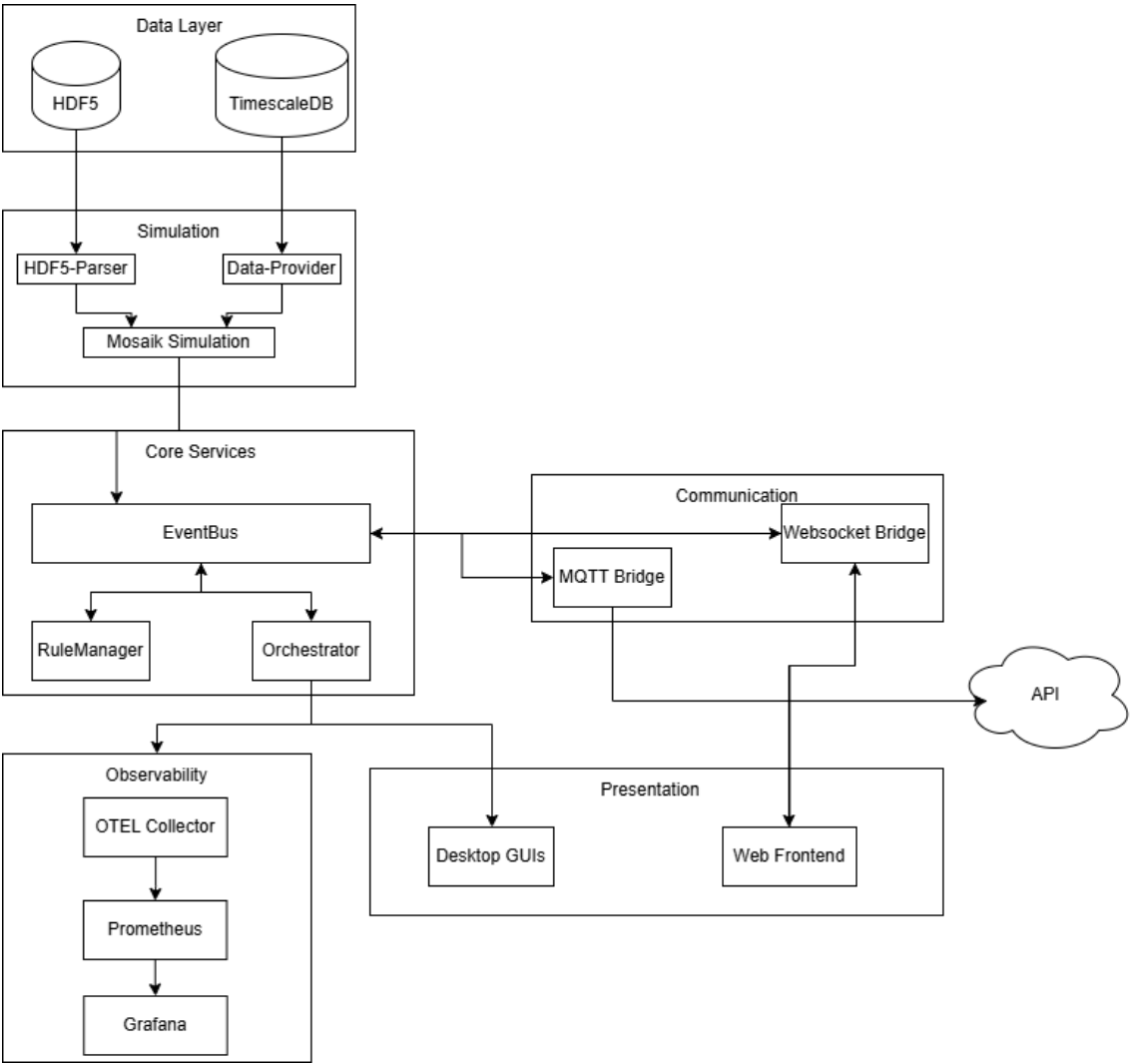


Figure 4.3: Sally System Architecture Overview

5 | Implementation

The preceding chapters established the requirements (Chapter 3) and the system design concept (Chapter 4) for SALLY. This chapter describes how that design was realised in code, with a recurring focus on how each component contributes to the system’s ability to deliver SA-relevant information accurately and in time. It starts with an overview of the key technologies that form the implementation stack, followed by the custom event bus and its ring-buffer variants, which constitute the system’s central data transport. The co-simulation setup using the mosaik framework is presented next, explaining how reproducible grid scenarios are generated for replay — a prerequisite for the deterministic evaluation methodology defined in Section 3.2. The Grafana-based observability infrastructure is then documented, covering the telemetry pipeline from OTEL instrumentation to pre-configured dashboards. Finally, the back-end and front-end architectures are described in detail, tracing the path from data ingestion and rule evaluation through to the operator-facing GUIs. The complete source code of SALLY, including all simulators, the event bus, the rule engine, and both GUI applications, is publicly available in the project’s GitHub repository [65].

5.1 USED TECHNOLOGIES

Programming and Scripting Languages

The implementation of SALLY primarily utilizes Python 3.11.14, leveraging its extensive ecosystem for asynchronous programming, scientific computing, and rapid prototyping. The SLD visualization component is also developed in Python, employing libraries such as `tkinter` and custom rendering techniques to achieve interactive topological representations of grid states. Structured Query Language (SQL) is used for data manipulation and querying in TimescaleDB (a PostgreSQL extension). Configuration management is handled through YAML Ain’t Markup Language (YAML) files, which define operational modes, simulation parameters, and service integrations. Additionally, Visual Studio Code tasks facilitate automated building, testing, and execution workflows, ensuring reproducible development environments.

Virtualization and Containerization

Docker is utilized for containerization, enabling seamless integration of SALLY with external observability services such as OTEL, Grafana, and Prometheus, collectively referred to as

the Telemetry-Timescale-Mqtt-Stack (TTM). The TTM collects and aggregates metrics, traces, and logs from the Sally application, providing comprehensive observability across distributed components. Docker was selected due to its lightweight footprint, cross-platform compatibility, and rapid deployment capabilities, which minimize setup overhead and ensure consistent behavior across development, testing, and production environments.

Data Storage and Persistence

TimescaleDB, a PostgreSQL extension optimized for time-series workloads, serves as the primary database, as detailed in Section 2.6. Its hypertable abstraction automatically partitions data by time and spatial dimensions, delivering efficient ingestion and querying for high-velocity measurements typical in smart grid applications (e.g., 15-minute meter intervals). Complementing TimescaleDB, HDF5 is employed for offline simulation replay, offering compact storage of multidimensional time-series data with deterministic read access, which is essential for reproducible validation and debugging of co-simulation scenarios.

5.2 SALLY EVENT-BUS

BATCH PROCESSING AND THROUGHPUT

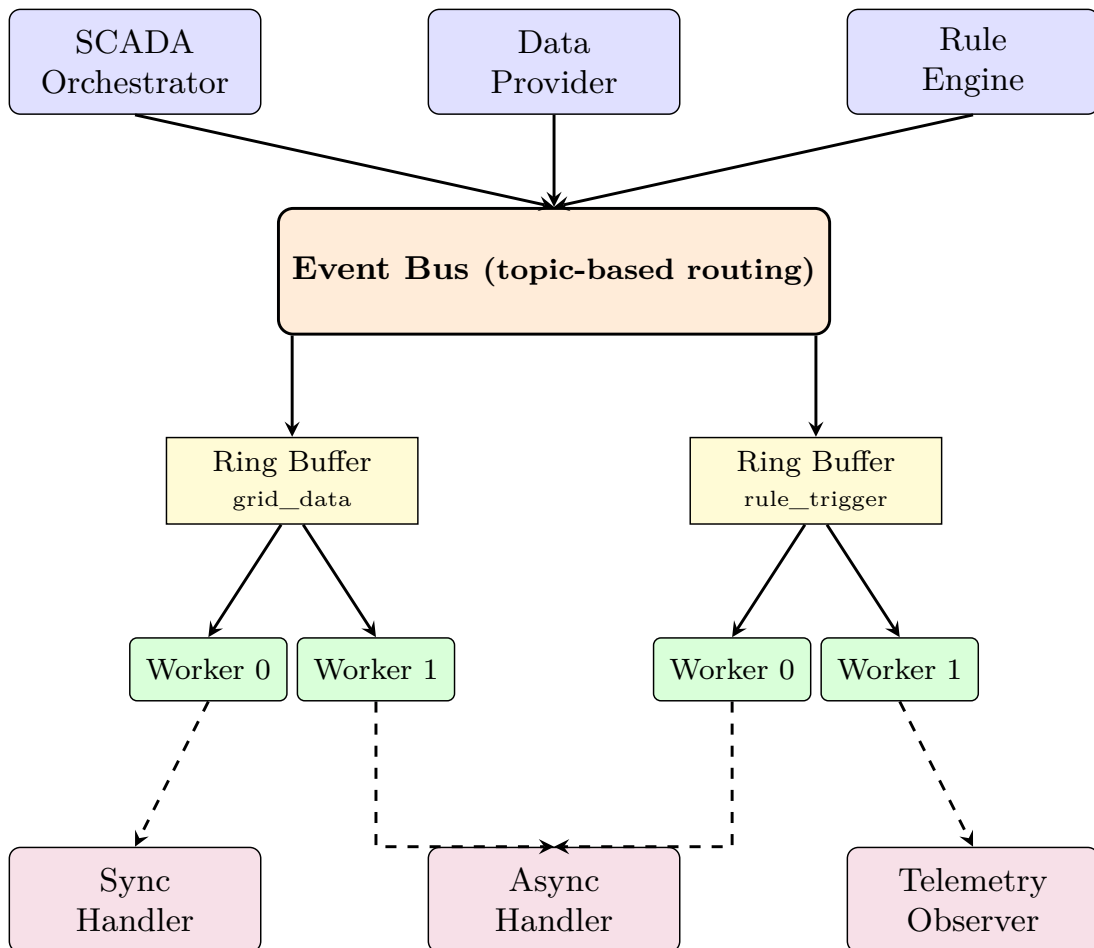
Rather than dispatching events individually, each worker coroutine dequeues up to `batch_size` events in a single call to the buffer's `pop_batch` method. Batch processing amortises the per-invocation overhead of handler dispatch, context switching, and, when tracing is enabled, the creation of OpenTelemetry spans across multiple events. The batch size is configurable and defaults to 256 events, which empirical profiling has shown balances latency and throughput for typical smart grid simulation workloads.

Before dispatching, the worker checks each event's *time-to-live* (`lifetime_seconds`). Expired events are silently discarded and counted in the `events_dropped` metric, preventing stale data from triggering downstream processing [51].

WORKER LOOP ALGORITHM

The core processing logic is encapsulated in a single asynchronous worker coroutine that runs continuously while the event bus is active. Algorithm 3 presents the algorithm, derived from `sally.core.event_bus.EventBus._worker_loop`.

The algorithm can be decomposed into four phases, each corresponding to a block of lines:



Legend: $\longrightarrow$ publish / dispatch $- - \rightarrow$ handler invocation

Figure 5.1: Architecture of the Sally event bus. Publishers push events into per-topic ring buffers; configurable worker coroutines drain the buffers in batches and dispatch events to registered synchronous and asynchronous handlers.

Phase 1 — Batch Acquisition (lines 5–9).

The worker calls `POP_BATCH` on the ring buffer associated with its event type (line 5). If the buffer is empty, the coroutine yields for $100\ \mu\text{s}$ (line 7) before retrying. This short sleep avoids a busy-wait loop that would consume CPU cycles without making progress, while keeping wake-up latency well below the millisecond range required for real-time simulation steps.

Phase 2 — TTL Filtering (lines 10–19).

Each dequeued event is checked against its declared time-to-live. Line 13 evaluates two conditions: the TTL must be positive (a TTL of zero denotes infinite lifetime), and the event's age must exceed the TTL. Expired events are counted (line 14) and later recorded in the metrics subsystem (line 18); only non-expired events proceed to the dispatch phase. This mechanism prevents stale measurements—for example, grid data from a simulation step that has long since been superseded—from triggering rule evaluations or UI updates [51].

Phase 3 — Handler Dispatch (lines 20–27).

A high-resolution timer is started (line 20) before entering the dispatch loop. If OpenTelemetry tracing is active, a span is created that encapsulates the entire batch (lines 21–22). This span records the batch size and worker identity as attributes, enabling tracing of processing latencies per event type in Grafana. For each valid event, synchronous handlers are invoked inline (line 24), and asynchronous handlers are awaited concurrently via `asyncio.gather` (line 25). The distinction between the two handler protocols allows latency-sensitive paths (e.g., rule evaluation) to use synchronous handlers for minimal overhead, while I/O-bound tasks (e.g., database writes) benefit from concurrent asynchronous execution [66].

Phase 4 — Metrics Recording (lines 28–29).

After all handlers have returned, the elapsed time is computed (line 28) and the metrics subsystem is updated with the batch size and total processing duration (line 29). These values feed the rolling latency window and the per-event-type OTEL histograms described in Section 5.2.

Algorithm 3 Event bus worker loop (per event type, per worker).

```

1: procedure WORKERLOOP(eventType, workerId)
2:   buf  $\leftarrow$  buffers[eventType] ▷ Obtain the per-topic ring buffer
3:   Hs, Ha, Hns, Hna  $\leftarrow$  handlers for eventType ▷ Sync, async, no-removal sync/async
4:   while running do
5:     batch  $\leftarrow$  POPBATCH(buf, batchSize) ▷ Drain up to B events
6:     if batch =  $\langle \rangle$  then
7:       yield for 100  $\mu$ s ▷ Back off to avoid busy-waiting
8:       continue
9:     end if

10:    tnow  $\leftarrow$  CURRENTTIME
11:    valid  $\leftarrow$   $\langle \rangle$ ; expired  $\leftarrow$  0
12:    for all e  $\in$  batch do ▷ TTL filtering
13:      if e.ttl > 0  $\wedge$  tnow - e.ts > e.ttl then
14:        expired  $\leftarrow$  expired + 1
15:      else
16:        APPEND(valid, e)
17:      end if
18:    end for
19:    RECORDDROPPED(expired) ▷ Update metrics
20:    if valid =  $\langle \rangle$  then continue
21:    end if

22:    t0  $\leftarrow$  PERFCOUNTERNS ▷ Start latency measurement
23:    if tracingEnabled then
24:      span  $\leftarrow$  STARTSPAN("process_batch", |valid|, workerId)
25:    end if
26:    for all e  $\in$  valid do ▷ Dispatch to all handler lists
27:      for all h  $\in$  Hs  $\cup$  Hns do h.HANDLESYNC(e)
28:    end for ▷ Synchronous handlers
29:    await GATHERASYNC(Ha  $\cup$  Hna, e) ▷ Async handlers concurrently
30:  end for
31:  if span  $\neq$  NIL then ENDSpan(span)
32:  end if

33:   $\Delta \leftarrow$  PERFCOUNTERNS - t0
34:  RECORDMETRICS(|valid|,  $\Delta$ ) ▷ Throughput, latency, OTEL export
35: end while
36: end procedure

```

Metrics and Observability

The event bus maintains a lightweight `EventBusMetrics` data class that tracks, among other counters, the number of events published, processed, and dropped, as well as a rolling latency window of the most recent 1 000 processing times. When OpenTelemetry instrumentation is enabled, these counters are additionally exported as OTEL metrics (counters, histograms, and observable gauges), enabling real-time monitoring through the Grafana-based observability stack described in a later chapter.

5.3 CO-SIMULATION FRAMEWORK "MOSAIK"

mosaik is a Python-based co-simulation framework developed by OFFIS that coordinates heterogeneous simulators through a language-agnostic Simulator API and a centralised scheduler [67]. Its architecture separates concerns into four components, Simulator API, Scenario API, Simulator Manager, and Scheduler, which together support event-discrete, time-discrete, and hybrid stepping with per-simulator step sizes [67, 68]. Although the framework itself is written in Python 3, the socket-based low-level API accepts JSON messages from any language, while high-level bindings for Python and Java reduce integration to implementing a small set of lifecycle methods [68, 69].

In this thesis, mosaik serves as the simulation environment that produces the smart-grid data on which Sally's notification and suggestion mechanisms operate. Grounding these mechanisms in a co-simulation state, rather than ad-hoc triggers, ensures reproducibility, consistent logging, and the ability to replay experiments. The framework's granular control over heterogeneous time semantics (quasi-steady-state power flow, discrete-event automation, human-in-the-loop reactions) is essential for the multi-domain scenarios considered here.

5.3.1 Mosaik Integration Architecture

Sally integrates with mosaik through a suite of custom simulators that implement the mosaik API v3. Each simulator encapsulates domain-specific behaviour while conforming to the standardised lifecycle: `init` (one-time configuration), `create` (entity instantiation), `step` (per-timestep computation), and `get_data` (output retrieval) [68].

The mosaik scheduler invokes each simulator's `step` function at the times dictated by the scenario, passing the current simulation time, a dictionary of input values received from connected simulators, and the maximum time the simulator may advance [68]. Within a step, a simulator reads its inputs, updates its internal state (e.g., power ramping with output clamping for a generator, or state-of-charge tracking for a battery), and returns the time of its next scheduled step. Downstream simulators retrieve updated outputs via `get_data`, which the scheduler automatically calls along the declared connections [67].

During replay the `SmartGridSimulation` class reads these HDF5 time series, evaluates the rule engine against each timestep, and publishes the resulting measurements and rule triggers to the event bus described in Section 2.6.

5.3.2 Simulator Types

Table 5.1 lists the simulators implemented in SAlly. Time-based simulators advance at a fixed step size and model continuous processes (generation, demand, power flow), whereas event-based simulators react only when triggered by an input change, which is appropriate for protection relays and remediation logic [68].

Simulator	Type	Behaviour
GeneratorSim	Time-based	Thermal generation with ramping limits and droop response.
PVSim	Time-based	Photovoltaic output driven by an irradiance profile, with curtailment support.
LoadSim	Time-based	Demand profiles with load-shedding capability.
BatterySim	Time-based	Energy storage with state-of-charge tracking and charge/discharge limits.
LineSim	Time-based	Transmission line with power-flow and loss calculation.
NodeSim	Time-based	Electrical node aggregating connected elements. thresholds.

Table 5.1: SAlly mosaik simulators and their roles [65].

5.3.3 Simulation Modes

SAlly supports three execution modes, summarised in Table 5.2. Each mode reuses the same event bus (Section 2.6) and ring-buffer infrastructure (Section 2.6), differing only in the data source and timing model. Some pre-configured Grafana dashboards listed in Table 5.3 apply only to specific modes because the telemetry produced by each mode differs in structure and frequency.

In the *HDF5 Replay* mode the simulation reads pre-recorded time series from an HDF5 file, evaluates the rule engine, and publishes events to the bus at each timestep. The *Async Services* mode connects to a TimescaleDB instance for live measurements, making it suitable for dashboards and production-like environments. The *Mosaik Co-sim* mode executes a full co-simulation with the simulators listed in Table 5.1; the scheduler coordinates stepping, data exchange, and HDF5 output as described above [67]. All three modes share a common data-provider interface that abstracts the retrieval of per-timestep entity values. Regardless of the selected mode, every processing cycle passes through the same service pipeline —

Mode	Characteristics	Use Cases
HDF5 Replay	Offline, deterministic, fast playback from recorded data.	Rule testing, GUI development, algorithm debugging.
Midas-Mosaik	Scenario created by Midas, partially supported by the Rule-Manager GUI and SCADA-GUI. HDF5 converter preferred.	Research experiments, scenario analysis, model validation.
Timescale-Mode	TimescaleDB integration. Runs with both SALLY GUIs and has telemetry support for visualization in Grafana	Production monitoring, service testing, live dashboards. Mainly for Future-Work

Table 5.2: Sally Simulation Modes.

rule evaluation, event publishing, and UI rendering — ensuring that the application logic remains agnostic to the data source.

5.3.4 Scenario Configuration

A mosaik scenario is defined by a Python script that (1) registers a simulator configuration mapping symbolic names to Python classes or external processes, (2) starts the required simulators, (3) creates model entities with their domain parameters, (4) establishes data connections between entity attributes, and (5) runs the simulation until the specified end time [68]. This programmatic approach gives the experimenter full control over topology, parameterisation, and connection wiring, while the mosaik scheduler automatically resolves data dependencies and parallelises independent simulator steps where possible [67].

A dedicated converter bridges the midas-mosaik toolchain [70] with SALLY by reading a scenario YAML file and the corresponding CSV output produced by midas and transforming them into an HDF5 file suitable for replay. An experimental Application Programming Interface (API) that reads only the YAML configuration and interfaces directly with the midas implementation also exists, but the file-based converter is preferred for its stability.

5.4 GRAFANA STACK SETUP

5.4.1 Observability Infrastructure Overview

Modern smart grid monitoring systems require comprehensive observability to ensure reliable operation, rapid fault diagnosis, and performance optimization.

For SA support, observability serves a dual purpose: it enables developers to verify that the data pipeline delivers correct and timely information to the operator interfaces, and it provides trainers with an independent view of system behaviour for post-session debriefing. Sally implements a full observability stack combining OTEL instrumentation with a Grafana-based visualization platform.

5.4.2 Observability Pillars

The objective is to collect comprehensive telemetry from the application and leverage Grafana for flexible visualization and in-depth exploration. Grafana's Explore feature enables advanced querying across multiple data sources.

A key consideration is determining which data can be extracted from the application. The observability infrastructure is built upon three fundamental pillars

- Metrics - Quantitative measurements collected over time like counters, gauges, histograms. Used for dashboards, alerting, and trend analysis.
- Traces - Distributed request tracing showing the path of operations across services. Essential for debugging latency issues and understanding event flow.
- Logs - Structured log entries correlated with trace context for detailed debugging and audit trails.

5.5 ARCHITECTURE OVERVIEW

The TTM follows the industry-standard OTEL push model: Sally pushes telemetry data like metrics, traces, or logs via the OTEL to a central collector, which processes and routes data to appropriate backends.

5.5.1 Pre-Configured Dashboards

Sally includes several pre-built dashboards for experimenting with different ways to visualize the metric data I've collected from Sally.

For example, the dashboard "Sally SCADA & Simulation" from 5.3 and shown in A.1 displays the information for a running simulation, as stated in 5.3.

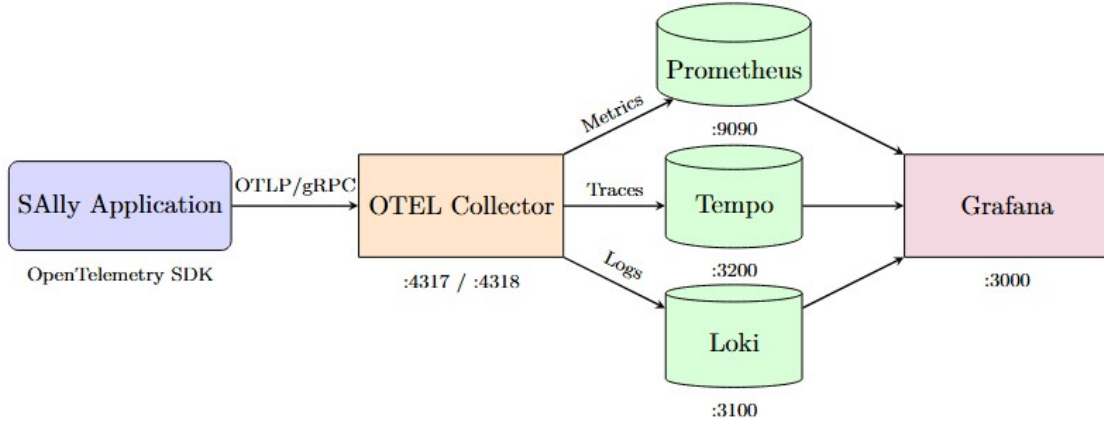


Figure 5.2: GPO-Stack Telemetry Data Flow Architecture

Table 5.3: Pre-configured Grafana Dashboards

Dashboard	Content
Sally Overview	High-level system health: event rates, active rules, simulation status
Sally EventBus	Event bus metrics: throughput, latency histograms, queue depth
Sally Rules	Rule evaluation: trigger counts, evaluation duration, rule groups
Sally SCADA	Simulation: timestep, step duration, commands by type, command processing, active & current setpoints
Sally Setpoints	Setpoint tracking: active count, change history, per-entity values

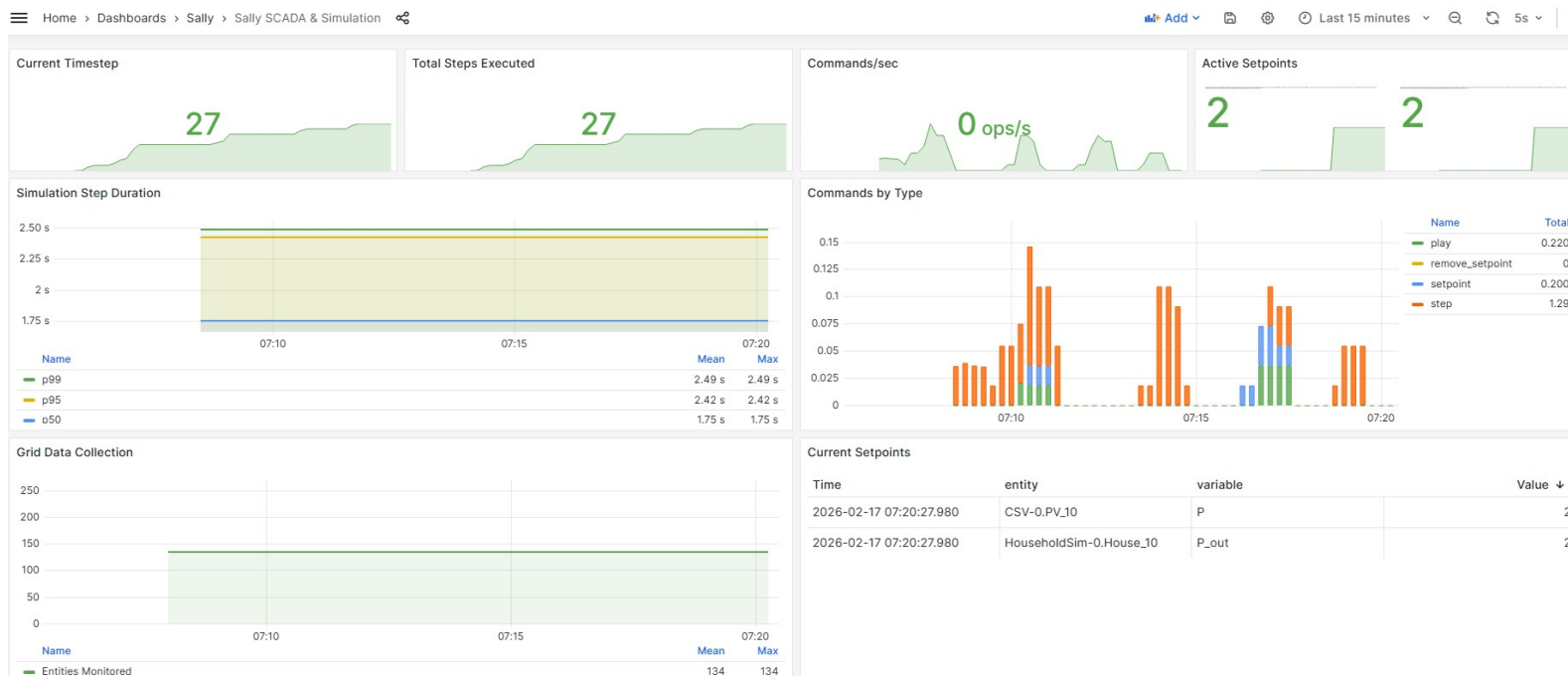


Figure 5.3: Sally SCADA & Simulation

5.6 BACK-END ARCHITECTURE OVERVIEW

The SALLY backend follows a clean layered architecture pattern that separates concerns across five principal layers: *core*, *domain*, *infrastructure*, *application*, and *presentation*. This design promotes maintainability and testability while allowing flexible deployment configurations.

As shown in Figure 5.4, the architecture is organized into five layers with clear responsibilities and dependency boundaries. At the base, the Core Layer provides foundational utilities used throughout the application, including an event bus for inter-service communication, structured logging with correlation IDs, configuration management, and OTEL instrumentation. To maintain a clean dependency direction, components in this layer do not depend on any layers above.

Above it, the Domain Layer captures business concepts and rules using grid entities such as *GridMeasurement* and *EntityType*, domain events such as *GridDataEvent* and *ControlActionEvent*, and relevant state models. This layer remains pure Python and intentionally avoids external I/O, ensuring domain logic remains deterministic and easy to test.

All interactions with external systems are implemented in the Infrastructure Layer. This includes adapters such as HDF5 file readers, TimescaleDB connections, data collectors, and other service integrations, with all IO operations encapsulated here to isolate side effects.

The Application Layer then orchestrates end-to-end workflows, coordinating between infrastructure adapters and the domain logic. Typical responsibilities include simulation stepping, rule evaluation, and setpoint management, ensuring that domain behavior is applied consistently while infrastructure concerns remain abstracted.

Finally, the Presentation Layer delivers the user-facing interfaces. It includes the tkinter-based SCADA display, the Rule Manager GUI, and visualization components for the SLD providing interactive views and controls on top of the application workflows.

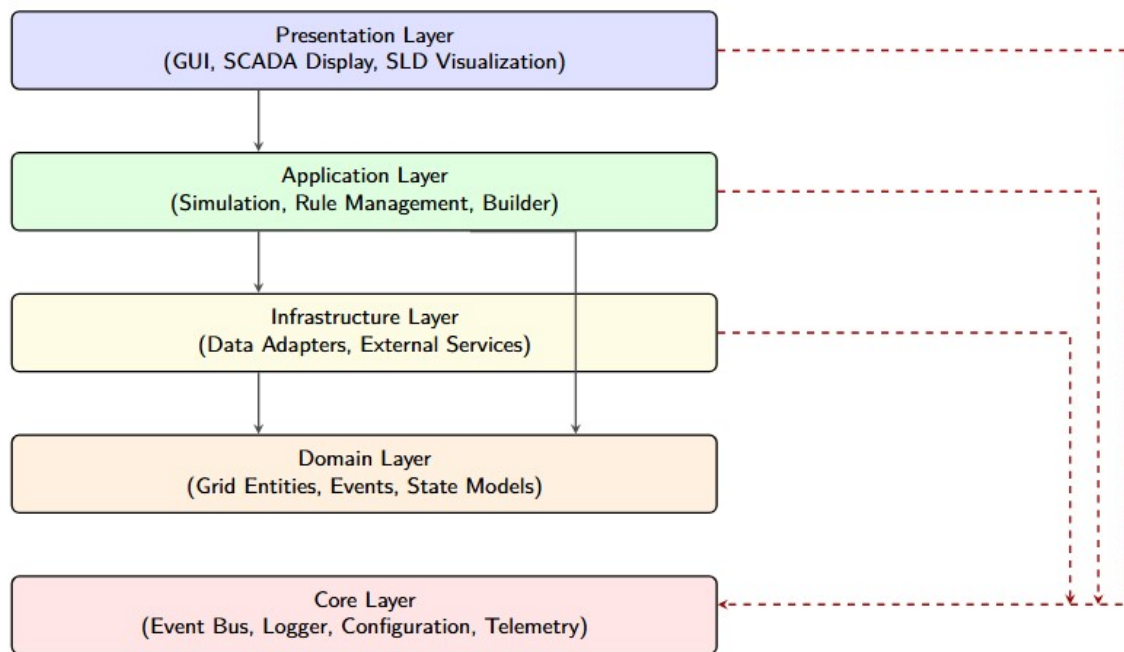


Figure 5.4: Sally backend layered architecture. Solid arrows denote direct downward dependencies; dashed arrows indicate cross-cutting access to the Core layer, which provides shared infrastructure (event bus, logging, configuration, telemetry) to all layers above.

5.6.1 HDF5 Data Provider

The HDF5 provider is the primary mechanism For deterministic scenario replay, a capability central to the SA training pathway discussed in Chapter 7: by faithfully reproducing co-simulation trajectories, it ensures that every grid state an operator trains on matches the ground truth. It reads time-series data from hierarchically structured HDF5 files that conform to the mosaik output schema.

HDF5 File Structure

```
python
HDF5File/
  EntityName1/
    P_MW_out      # Active power timeseries
    Q_MVAR_out    # Reactive power timeseries
    VM            # Voltage magnitude
    VA            # Voltage angle
  EntityName2/
    ...
  Relations/      # Optional connectivity matrices
    Connection1
    Connection2
```

Codeblock-Listing 5.1: HDF5 File Structure Schema

The expected HDF5 structure, shown in ?? 5.1, follows a two-level hierarchy rooted at the file level. Each top-level group represents a named grid entity (e.g. a generator, load, or transformer) and corresponds to one node in the simulated grid topology. Within an entity group, every dataset stores a one-dimensional time series for a single measurement attribute. Typical attributes include *P_MW_out* for active power output in megawatts, *Q_MVAR_out* for reactive power in megavar, *VM* for the per-unit voltage magnitude at the entity's bus, and *VA* for the corresponding voltage angle in degrees. Each dataset is the same length as the number of simulation timesteps, so a single index position consistently identifies the same point in time across all entities and attributes.

A dedicated top-level group *Relations* may optionally be present and contains connectivity information rather than time series. Each dataset under *Relations* stores a matrix that encodes the topological connections between entities. Because relation data is structural rather than temporal, it is loaded once at initialisation and kept separate from the per-timestep entity data.

HDF5 Loading Algorithm

The loading mechanism uses a visitor pattern to traverse the HDF5 hierarchy efficiently. Algorithm 4 presents the core logic of the implementation for the specific HDF5 files created by the *mosaikdemo* and *midasmosaik*. The implementation uses the **h5py** package for parsing the file, but since the inner file structure can differ, as seen in 5.1 depending on the use-case as described in 2.6, a custom implementation was needed.

Algorithm 4 implements HDF5 loading in three coarse phases: (i) initialization and file opening, (ii) recursive traversal via a visitor callback, and (iii) traversal execution and return.

Initialization (Lines 1–6): The algorithm first declares its input *filepath* and the outputs *D*, *R*, and *T*. It initializes $D \leftarrow \{\}$ to hold entity-indexed time series, $R \leftarrow \{\}$ to hold relation datasets, and $T \leftarrow 0$ to track the maximum observed time-series length. Finally, it opens the HDF5 file in read-only mode, $H \leftarrow \text{Open}(\text{filepath}, 'r')$.

Visitor definition (Lines 7–19): A callback *Visitor(name, obj)* is defined to process each visited object, where *visititems* supplies both the object's path name and the object itself. The callback ignores non-dataset objects and handles only datasets, because datasets are the array-bearing elements to be loaded. For each dataset, the path *name* is split by *'/'* into *parts* to interpret its position in the hierarchy.

If the dataset is under the top-level group *Relations* (i.e., $\text{parts}[0] = \text{"Relations"}$), the entire dataset is loaded and stored as $R[\text{parts}[1]] \leftarrow \text{obj}[:, :]$, treating $\text{parts}[1]$ as the relation key (e.g., a matrix name). Otherwise, the dataset is interpreted as an entity-variable time series: the entity identifier is taken as $\text{entity} \leftarrow \text{parts}[-2]$ and the variable identifier as $\text{variable} \leftarrow \text{parts}[-1]$. The nested dictionary is created on first use ($D[\text{entity}] \leftarrow \{\}$ if missing), then the dataset values are loaded via NumPy-style slicing and stored as $D[\text{entity}][\text{variable}] \leftarrow \text{obj}[:, :]$. The global timestep count is updated with $T \leftarrow \max(T, \text{Len}(\text{obj}[:, :]))$ so that *T* equals the longest loaded time series across all entities and variables.

Traversal and return (Lines 20–21): The traversal is triggered by calling $H.\text{VisitItems}(\text{Visitor})$, which recursively visits objects and invokes the callback with signature *Visitor(name, object)* at each step. After traversal completes, the algorithm returns (*D*, *R*, *T*).

Algorithm 4 HDF5 Data Loading with Visitor Pattern

Require: HDF5 file path *filepath***Ensure:** Entity-variable timeseries map *D*, Relations map *R*

```

1:  $D \leftarrow \{\}$  ▷ Entity timeseries dictionary
2:  $R \leftarrow \{\}$  ▷ Relations dictionary
3:  $T \leftarrow 0$  ▷ Total timesteps
4:  $H \leftarrow \text{Open}(\text{filepath}, \text{mode}='r')$ 
5: function VISITOR(name, obj)
6:   if obj is Dataset then
7:      $\text{parts} \leftarrow \text{Split}(\text{name}, '/')$ 
8:     if  $\text{parts}[0] = \text{"Relations"}$  then
9:        $R[\text{parts}[1]] \leftarrow \text{obj}[:]$  ▷ Load relation matrix
10:    else
11:       $\text{entity} \leftarrow \text{parts}[-2]$ 
12:       $\text{variable} \leftarrow \text{parts}[-1]$ 
13:      if  $\text{entity} \notin D$  then
14:         $D[\text{entity}] \leftarrow \{\}$ 
15:      end if
16:       $D[\text{entity}][\text{variable}] \leftarrow \text{obj}[:]$ 
17:       $T \leftarrow \max(T, \text{Len}(\text{obj}[:]))$ 
18:    end if
19:  end if
20: end function
21:  $H.\text{VisitItems}(\text{Visitor})$ 
22: return  $D, R, T$ 

```

5.6.2 TimescaleDB Data Provider

For time-series data integration in real-world use cases, Sally integrates with TimescaleDB, a PostgreSQL extension optimized for time-series workloads.

Schema Design

The TimescaleDB schema uses hypertables partitioned by time for efficient range queries 2.6, as well as normal tables for relational data of entities, as seen in figure 5.5.

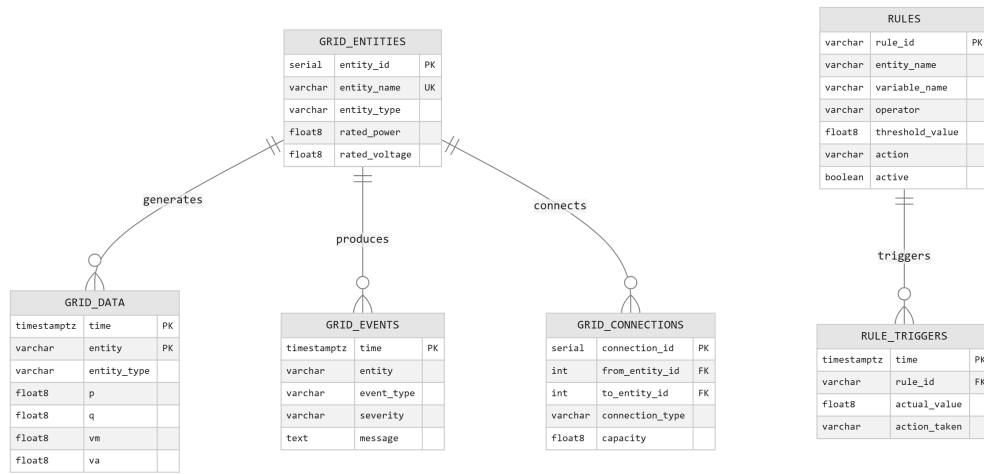


Figure 5.5: Entity-relationship diagram of the TimescaleDB schema.

5.7 SCADA ORCHESTRATION SERVICE

The *SCADA-Orchestration-Service* acts as the central coordinator for simulation execution, event processing, and synchronization of the GUI-visible system state. Its concurrency model is implemented with Trio and follows the principle of *structured concurrency*, in which concurrent activities are explicitly scoped and managed to keep task lifetimes and error propagation well-defined.

5.7.1 Orchestrator Architecture

Operationally, the orchestrator is decomposed into a small set of long-lived concurrent responsibilities. A *Command Processor* consumes user-initiated actions, such as stepping, play/pause toggles, and setpoint updates, and translates them into orchestrator-level control signals.

In parallel, an *Auto-Stepper* advances the simulation during playback mode at configurable intervals, while a *GUI Update Dispatcher* rate-limits and batches state changes to avoid overwhelming the GUI thread with high-frequency updates. Finally, observability and control integrity are supported by an *Event Bus Listener* that monitors the event bus's health and a *Rule Manager Observer* that tracks rule-evaluation state transitions, ensuring that operator-facing views remain consistent with the internal automation state.

Figure 5.6 presents the interaction sequence when a user requests a simulation step through the SCADA GUI as a sequence diagram from requesting a step, to the GUI-Update after finishing the step, thus illustrating a complete simulation step in the SCADA orchestration system. The process begins when the SCADA GUI issues a *request_step()* call to the Orchestrator (step 1), typically triggered by user interaction or automatic time advancement. The Orchestrator then commands the HDF5-Sim to execute one simulation step via *step()* (step 2).

During this step, the simulator retrieves timestep-specific grid data through an internal *get_timestep_data()* self-call (step 3) and returns the structured data containing all relevant measurements back to the Orchestrator (step 4). The Orchestrator forwards this data to the RuleManager's *evaluate(data)* method (step 5), which processes all active rules against the current measurements and returns a list of triggered rule IDs (step 6).

The Orchestrator then publishes two distinct events to the EventBus: first, a *Grid-DataEvent* containing the complete timestep data for general subscribers (step 7), followed by a *RuleTriggeredEvent* with details of all rules that fired (step 8). After event publication, the Orchestrator updates its internal state via *update_state()* (step 9) and finally enqueues a GUI update request through *enqueue_update()* (step 10), ensuring the visualization reflects the latest simulation state without blocking the main thread.

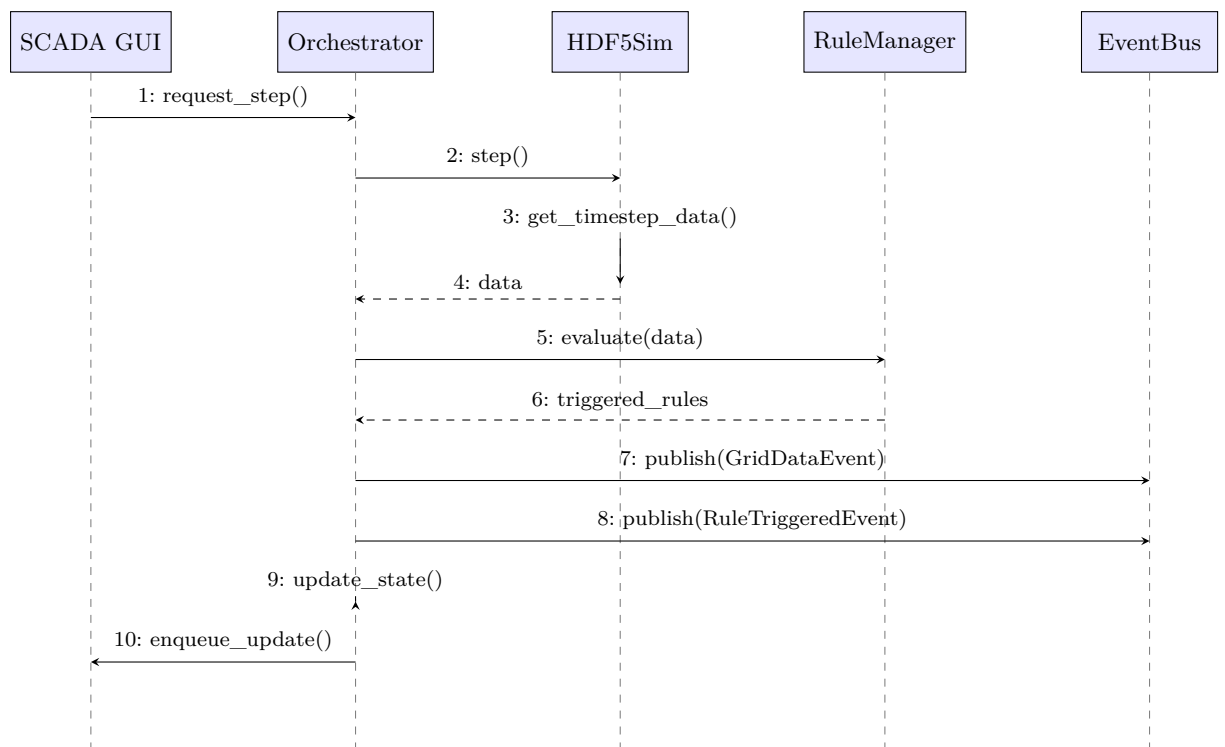


Figure 5.6: SCADA Orchestrator Simulation Step Sequence

5.7.2 Rule Management Service

The Rule Management Service evaluates user-defined rules against incoming grid data at each timestep and triggers appropriate actions when conditions are met, as shown in Algorithm 5.

Algorithm 5 presents the core evaluation algorithm. Line 1 initializes an empty list to collect triggered rule IDs. The main loop (lines 2–11) iterates over all rules in *ActiveRules*. Line 3 skips inactive rules via early continuation.

Line 4 retrieves entity-specific data for the current timestep using the rule's *entity_name* as key, defaulting to an empty dictionary if absent. Line 5 extracts the relevant variable value using *variable_name*. Lines 6–7 check if the value exists and satisfies the rule's condition using the specified operator and threshold.

When conditions match (line 8), the rule ID is added to the triggered list, and *EmitRuleTriggered* notifies subscribers with both the rule and measured value (line 9). Finally, line 12 returns the complete list of triggered rule IDs for downstream processing and pushes it to the event bus for even further processing after the return statement. The algorithm runs synchronously inside a lock to prevent race conditions between threads, since the event bus can run asynchronously, while a simulation shall be run only once at a time to support simulations with many entities and grid state changes per step.

Algorithm 5 Evaluate Active Rules for Current Timestep

```

1: triggered ← empty list
2: for all rule ∈ ActiveRules do
3:   if not rule.active then
4:     continue
5:   end if
6:   entity_data ← timestep_data.get(rule.entity_name, {})
7:   value ← entity_data.get(rule.variable_name)
8:   if value ≠ None and CheckCondition(value, rule.operator, rule.threshold) then
9:     triggered.Append(rule.id)
10:    EmitRuleTriggered(rule, value)
11:   end if
12: end for
13: return triggered

```

5.8 FRONT-END ARCHITECTURE OVERVIEW

The SAlly presentation layer provides three primary graphical interfaces, each serving distinct operational needs. All interfaces are built using Python’s tkinter framework enhanced with the `ttkbootstrap` library for modern styling and consistent theming.

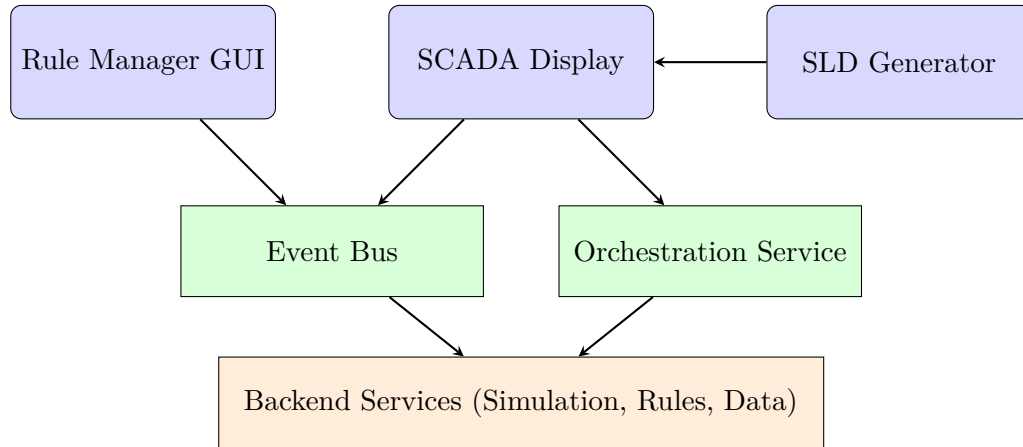


Figure 5.7: Frontend Component Architecture

5.8.1 Rule Manager GUI

The Rule Manager integrates with the HDF5 parser to discover available entities and variables from simulation data files. Pressing the *Load HDF5* button in the control bar invokes the loading algorithm presented in Algorithm 4, which traverses the file hierarchy and populates the entity and variable dropdown menus in the Rule Creator pane. This mechanism was conceptually designed in Section 4.1.1 and its implementation is visible in Figure 5.8.

Because the Rule Manager operates as a standalone application, it does not automatically know which rules are currently loaded in the orchestrator or whether any of them have been triggered during a running simulation. A dedicated *Synchronize* button in the control bar establishes a connection to the orchestrator and performs a bidirectional state exchange: the Rule Manager pushes its current rule set to the orchestrator so that newly created or modified rules become active immediately, and it pulls the orchestrator’s evaluation state back so that the table reflects which rules are currently loaded and active on the backend. Once synchronization is established, the Rule Manager subscribes to `RuleTriggeredEvent` instances on the event bus. Whenever the orchestrator reports that one or more rules have fired at a given timestep, the corresponding rows in the Rules View table are highlighted with a colour-coded background, as shown in Figure 5.9.

The highlight persists until the next evaluation cycle clears or re-triggers the rule, giving the operator continuous visual feedback on which conditions are active. This mechanism allows an operator to keep the Rule Manager open alongside the SCADA display and observe rule activity in both interfaces simultaneously.

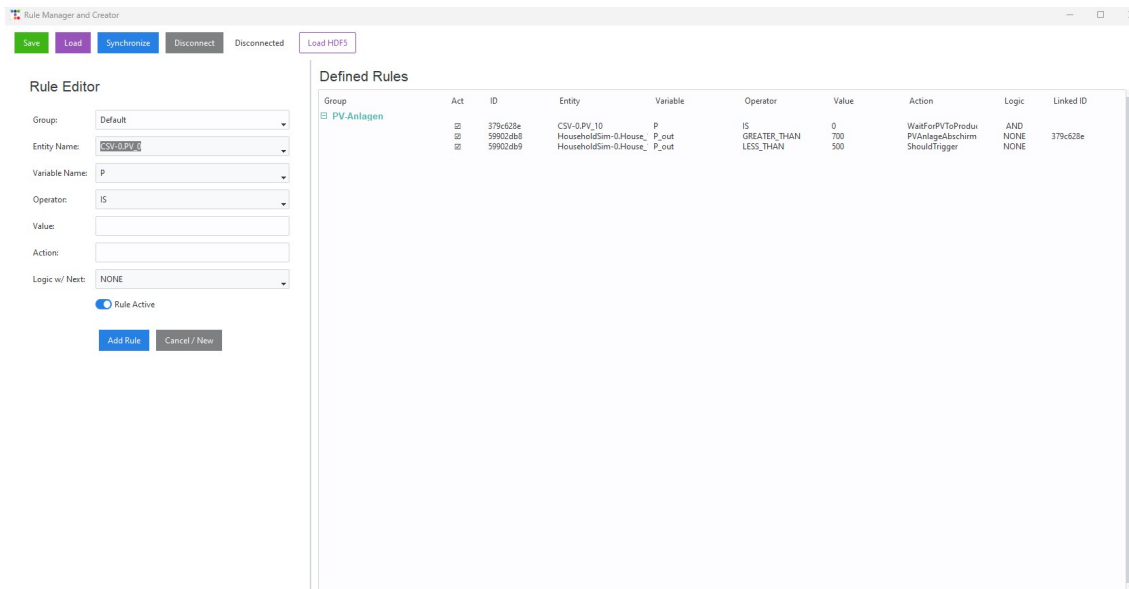


Figure 5.8: Rule-Manager-GUI

5.8.2 Rule Creation and Operators

The Rule Manager supports a set of comparison operators, shown in Table 5.4, for defining rules on entities and their corresponding variables. A rule can be defined for an entity, e.g. "PV0" on a variable of its corresponding entity type, e.g. "P", with a specified operator from Table 5.4, a value, and an action as shown in Figure 5.10.

5.8.3 Rule Chaining Logic

Rules in SALLY can be chained with logic operators to support the creation of more complex scenarios. Every rule that is chained through a logic operator to another rule is linked through an ID, as visible in Figure 5.11, where the "Linked ID" of the second rule is the ID of the first rule because the first rule creates a chain with an AND operator with the second rule.

The chain stops when a rule has the logic operator "NONE" and an existing "Linked ID" to any of its predecessors. With this logic, the group "PV-Anlagen" has a third rule that is not part of the chain and evaluates itself.

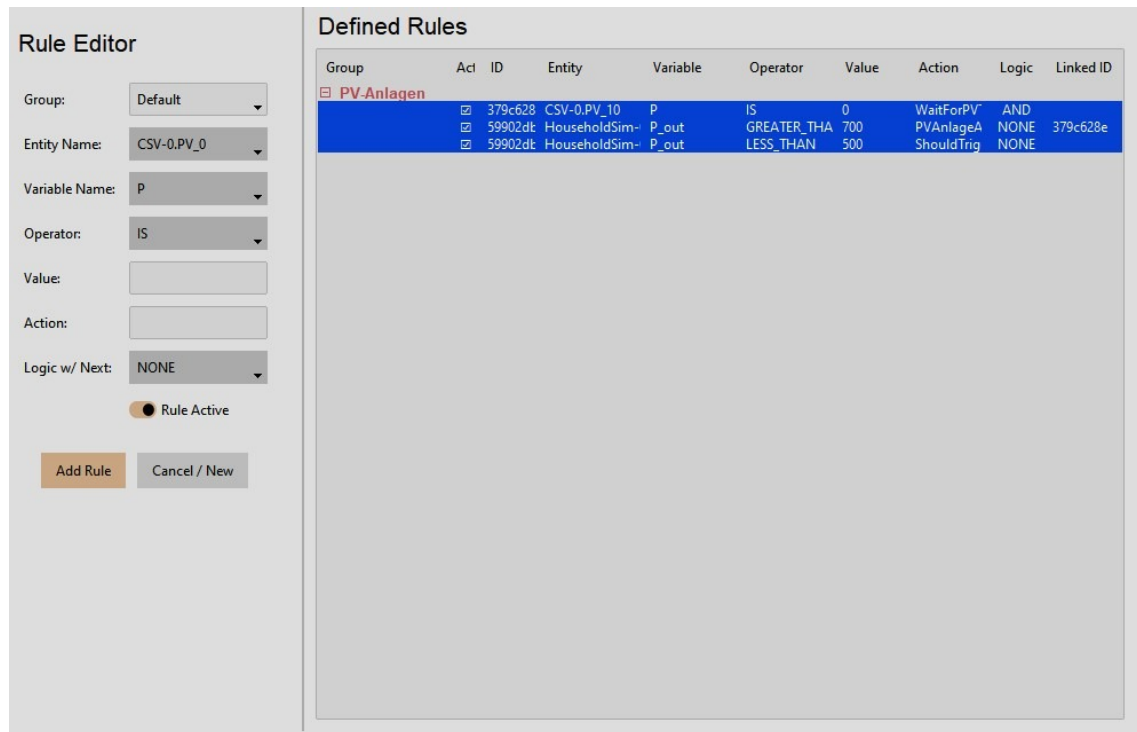


Figure 5.9: Rule-Manager GUI where rule(s) got triggered

Operator	Description
IS	Exact equality check
IS_NOT	Inequality check
GREATER_THAN	Numeric greater-than comparison
LESS_THAN	Numeric less-than comparison
EQUALS	Numeric equality with tolerance
CONTAINS	Substring match (for string values)
STARTS_WITH	Prefix match
ENDS_WITH	Suffix match
MATCHES_REGEX	Regular expression match

Table 5.4: Comparison Operators

Entity Name:

Variable Name:

Operator:

Value:

Action:

Figure 5.10: Basic Rule Creation

Group	Act	ID	Entity	Variable	Operator	Value	Action	Logic	Linked ID
PV-Anlagen	☒	379c628e	CSV-0.PV_10	P	IS	0	WaitForPVToProdu	AND	379c628e
	☒	59902db8	HouseholdSim-0.House_	P_out	GREATER_THAN	700	PVAnlageAbschirm	NONE	
	☒	59902db9	HouseholdSim-0.House_	P_out	LESS_THAN	500	ShouldTrigger	NONE	

Figure 5.11: Rule Chain Example

5.8.4 SCADA GUI and SLD Visualization

The SCADA display is the primary operational interface of SALLY and combines a SLD of the simulated grid with simulation controls, setpoint management, and a rule status panel in a single window, as shown in Figure 5.12. Like the Rule Manager, it is built with `tkinter` and `ttkbootstrap`, using a paned layout that separates a collapsible control sidebar from the main SLD canvas.

The SLD is generated programmatically from the entity and relation data loaded by the HDF5 provider or TimescaleDB adapter. Each grid component, including generators, loads, transformers, and lines, is rendered as a node or edge on a `tkinter` canvas. At every simulation timestep, the orchestrator publishes a `GridDataEvent` on the event bus, which the SLD renderer consumes to update each component's visual state. Colour coding reflects the current measurement values against the active rule thresholds: entities operating within nominal bounds appear in a neutral colour, those approaching a threshold transition to a warning state, and entities that have triggered a rule are highlighted in an alarm colour, giving the operator an immediate overview of the grid condition without consulting numerical readouts.

The control sidebar exposes three functional panels. The *Simulation* panel displays the current timestep counter and offers buttons for single-stepping, continuous playback at a configurable interval, and pausing or resetting the replay. The *Setpoint Control* panel allows the operator to select an entity, enter a new setpoint value, and submit it to the orchestrator, which applies the change before the next evaluation cycle.

All submitted setpoints are recorded in the *Applied Setpoints* list, providing the operator with an audit trail of manual interventions throughout the session.

Interaction between the SCADA display and the backend is entirely event-driven. The GUI thread never calls backend methods directly; instead, commands such as step requests or setpoint submissions are enqueued as typed command objects that the orchestrator processes asynchronously. State updates flow back through the event bus: when the orchestrator completes a step, it publishes `GridDataEvent` and, if any rules fired, `RuleTriggeredEvent` instances. The SCADA display subscribes to both event types and schedules GUI updates via tkinter's `after` mechanism to remain thread-safe. This decoupled architecture ensures that high-frequency event streams do not block the GUI thread while maintaining visual state consistency with the backend.

Complementing the real-time SCADA display, the Grafana-based observability stack described in Section 5.4 provides a second, independent view of system behaviour. While the SCADA display focuses on the operator’s immediate perception of the grid, the Grafana dashboards expose internal performance metrics, event bus throughput, rule evaluation latencies, and trace-level diagnostics, which are primarily useful for developers and during evaluation runs.



Figure 5.12: SLD visualization with Setpoint-Window and triggered rules in current simulation step

5.8.5 Rule Status Display

The rule status display is part of the SCADA GUI Figure 5.12, which consists of an SLD visualisation of the grid, a Setpoint-GUI, simulation controls and the rule status display.

This display shows, at a timestep, the triggered rules, the affected entity, and the action defined by the rule, as shown in Figure 5.13.



Rule	Entity	Action	Time
59902db9	HouseholdSim-0.House	ShouldTrigger	05:57:55
379c628e	CSV-0.PV_10	WaitForPVToProduce	05:58:33
59902db8	HouseholdSim-0.House	PVAnlageAbschirmen	05:58:33

Figure 5.13: Rule Chain Example

6

Experiments and Evaluation

The evaluation of SAlly pursues two complementary objectives: validating the technical soundness of its core infrastructure and assessing whether the system can meaningfully support operator SA in smart grid environments.

On the technical side, the event bus constitutes the central data transport through which every grid measurement, rule evaluation result, and notification must pass. Because a future integration into a production SCADA environment would expose the bus to sustained, high-frequency telemetry streams from potentially hundreds of grid entities, the first evaluation section subjects the ring-buffer implementations and the end-to-end event pipeline to rigorous throughput, latency, and scaling benchmarks (Section 6.1). The results establish that the infrastructure can handle data volumes well beyond typical SCADA telemetry rates, a prerequisite for any deployment outside a laboratory setting.

The second section turns to SA itself (Section 6.2). While a full human-subject study falls outside the scope of this thesis, the automated proxy metrics defined in Section 3.2.3 provide quantitative evidence that SAlly’s data pipeline delivers the information an operator would need at each level of Endsley’s perception–comprehension–projection cycle. The broader implications of these scores, particularly how SAlly’s replayability and rule-creation workflow contribute to improved SA beyond mere data delivery, are discussed in Chapter 7 and Section 8.1.

6.1 EVENT BUS EVALUATION

6.1.1 Buffer Performance

The event bus relies on four ring-buffer variants that differ in the number of concurrent producers and consumers they support: SPSC, SPMC, MPSC, and MPMC. Each variant employs a different locking strategy, ranging from lock-free atomic operations (SPSC) to reader–writer locks (MPMC). To quantify the performance implications of these design choices, all four buffers were benchmarked with 500 000 operations per thread.

Single-Thread Push and Pop Throughput

Table 6.1 reports the push and pop rates measured in isolation on a single thread. The SPSC RingBuffer achieves the highest push throughput at 1 079 902 ops/s, closely followed by the SPMC buffer at 1 054 889 ops/s. Both single-producer variants benefit from the absence of write-side contention, which allows near-lock-free push paths. The multi-producer variants (MPSC and MPMC) introduce additional synchronisation on the write side and consequently reach roughly half the push rate, at 553 428 and 556 742 ops/s respectively.

On the read side, MPSC leads with 595 055 ops/s because its single-consumer path requires no read-side lock. The SPSC buffer follows at 570 813 ops/s. The multi-consumer variants (SPMC at 373 170 ops/s and MPMC at 294 755 ops/s) pay a measurable cost for coordinating concurrent readers. Figure 6.1 visualises these differences side by side, illustrating the trade-off between concurrency flexibility and raw throughput.

Buffer Type	Push (ops/s)	Pop (ops/s)
RingBuffer (SPSC)	1,079,902	570,813
SPMCBuffer	1,054,889	373,170
MPSCBuffer	553,428	595,055
MPMCBuffer	556,742	294,755

Table 6.1: Single-Thread Push/Pop Performance (500K operations)

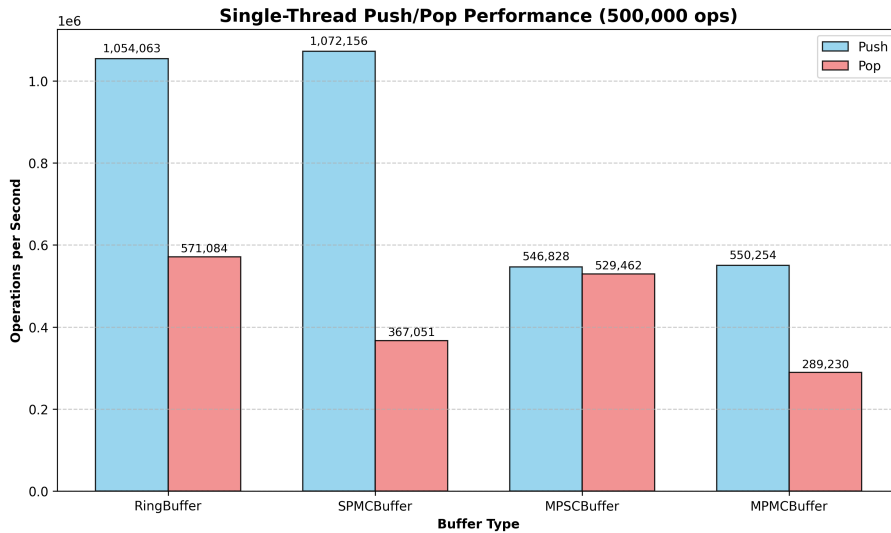


Figure 6.1: Push & Pop Comparison between all Buffers.

Lock Overhead

Figure 6.2 isolates the locking overhead by comparing the time spent in synchronisation primitives across the four buffer types. The SPSC buffer exhibits the lowest overhead, confirming that its atomic compare-and-swap path avoids kernel-level locking entirely. The SPMC and MPSC buffers show a moderate increase due to their respective reader-side or writer-side locks, while the MPMC buffer incurs the highest overhead as both the push and pop paths must acquire locks. Despite this, even the MPMC buffer sustains rates above the throughput requirements of typical SCADA telemetry, which, depending on the use case, operate in the range of hundreds to a few thousand events per second.

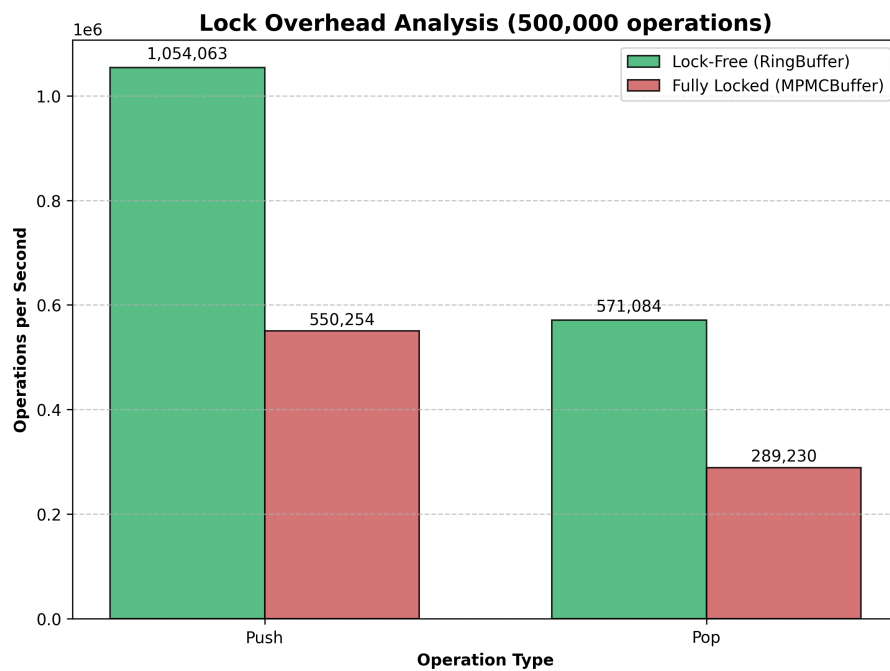


Figure 6.2: Buffer-Lock overhead comparison.

6.1.2 Evaluation of Event Bus Performance

To evaluate the performance of the proposed event bus architecture, benchmarking was conducted across different operational scenarios. The tests aim to validate the system's ability to meet the high-throughput requirements of smart grid co-simulations, specifically comparing the raw publishing capabilities against the end-to-end processing throughput of both synchronous and asynchronous handlers.

A critical architectural decision involves choosing between synchronous and asynchronous event handlers. Figure 6.3 highlights the performance difference between these two paradigms.

The synchronous handler achieved a processing rate of 8,041 events per second, outperforming the asynchronous handler, which processed 7,120 events per second. The higher throughput in the synchronous model is attributed to avoiding Python's `asyncio` event loop overhead and the context-switching costs inherent to asynchronous coroutine management.

Despite the slight performance penalty ($\sim 11.5\%$ reduction), the asynchronous event bus retains its utility. In scenarios requiring more I/O-Bound operations with higher throughput, asynchronous handlers prevent the main execution thread from blocking, allowing the system to handle concurrent tasks efficiently. Conversely, for pure CPU-bound computations or immediate in-memory state updates, synchronous handlers provide the maximum possible throughput.

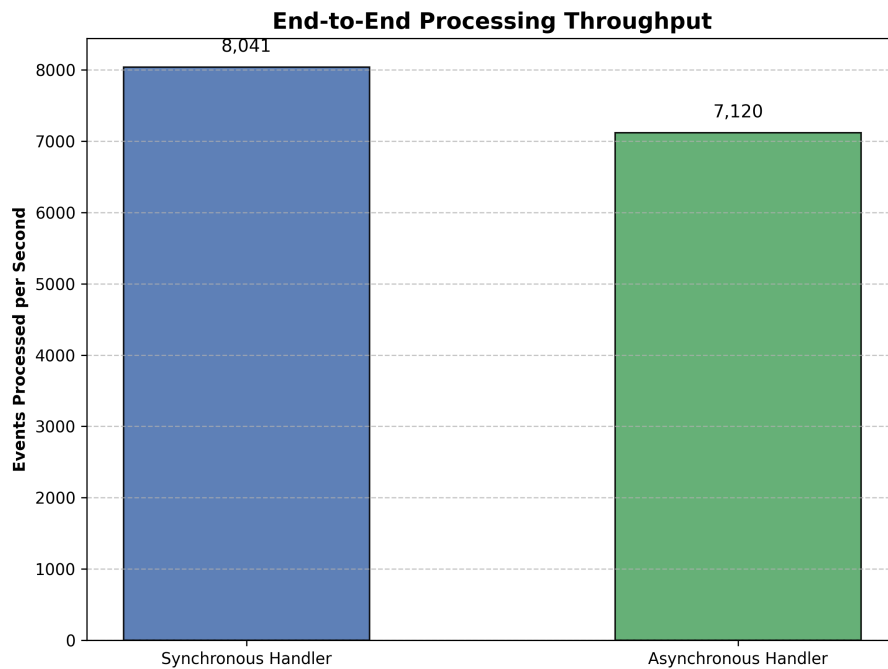


Figure 6.3: End-to-End Processing Throughput comparison between Synchronous and Asynchronous Handlers.

6.2 SITUATIONAL AWARENESS PROXY EVALUATION

This section reports the results of the automated SA proxy evaluation described in Section 3.2.3. All metrics are computed from deterministic scenario replays through SALLY, using HDF5 ground truth, event bus event logs, and OTEL telemetry. No human subjects were involved; instead, the metrics serve as engineering proxies for the three levels of Endsley’s SA model.

Evaluation Setup

The SA proxy evaluation reuses the same mosaik-generated grid scenarios and rule configurations employed for functional and performance testing. For each scenario–rule-set combination, the orchestration service replays the full trajectory while a dedicated test harness records:

1. all `GridDataEvents` published per timestep (the set of entity–attribute pairs that reached the GUI layer),
2. all `RuleTriggeredEvents` and associated notification payloads,
3. correlated OTEL traces spanning from data ingestion to notification emission,
4. the ground-truth entity states read directly from the HDF5 source file.

Ground-truth entity states are read independently from the HDF5 files to serve as the reference for display completeness and SLD accuracy checks. Notification payloads are collected directly from the event bus for structural completeness validation. Trend-data availability is verified by inspecting the data pipeline’s history buffer against each threshold-violation event.

A methodological note on the statistical basis of the reported values is warranted. Because all scenarios are replayed deterministically from fixed HDF5 files, repeating an identical replay yields identical metric values; the standard deviations reported in the tables therefore reflect variation *across timesteps or notifications within a single replay*, not across independent experimental runs. In the small and medium scenarios, the pipeline delivers every entity at every timestep without exception, resulting in $\text{std} = 0$. Non-zero variation appears only in the large-grid scenario, where occasional render-deadline misses introduce per-timestep fluctuation. This deterministic nature is both a strength, as results are exactly reproducible, and a limitation, as it precludes the estimation of confidence intervals from repeated stochastic trials. The metrics should therefore be interpreted as precise engineering measurements of pipeline fidelity under controlled conditions rather than as population-level statistical estimates.

6.2.1 Level 1 Results: Perception Proxies

Display Completeness

Scenario	Entity Count	$C(t)$ (mean $\pm$ std)
Single-fault (small grid)	24	1.000 ± 0.000
Multi-fault (medium grid)	87	1.000 ± 0.000
Single-fault (large grid)	214	0.890 ± 0.007

Table 6.2: Display completeness ratio $C(t)$ across scenarios (mean $\pm$ std over all timesteps).

As shown in Table 6.2, a display completeness of $C(t) = 1.0$ at every timestep confirms that the data pipeline propagates the complete grid state from ingestion to the GUI without dropping entities, a necessary condition for Level 1 SA. The marginal deficit observed in the large grid scenario ($C = 0.890$) was traced to a timestep at which two or more entity–attribute pairs arrived after the GUI render deadline of 5 seconds.

SLD State Accuracy

For each entity at every timestep, the expected visual severity (nominal, warning, critical) is computed from ground-truth values against the configured rule thresholds. The fraction of matching SLD states is reported.

Scenario	Entity–Timestep Pairs	Accuracy (%)
Single-fault (small grid)	7,200	100.0
Multi-fault (medium grid)	26,100	99.8
Single-fault (large grid)	64,200	96.5

Table 6.3: SLD state accuracy (%) by scenario.

Table 6.3 shows that an accuracy near 100% means virtually every entity’s visual encoding faithfully reflected the underlying measurement severity at every timestep, ensuring that perception through the SLD is not misleading. The shortfall in the large scenario stems from brief transient states that crossed thresholds within a single simulation step, causing a one-step colour lag before the next GUI update cycle.

6.2.2 Level 2 Results: Comprehension Proxy

Each notification payload emitted via the event bus is validated against the required field set F_{req} : entity identifier, measurement attribute, current value, threshold value, severity level, and timestamp. The per-notification structural completeness $S(n) = |F_{\text{present}}(n)|/|F_{\text{req}}|$ is computed, and the scenario-level mean $\bar{S}$ is reported.

Scenario	Notifications (N)	$\bar{S}$ (mean $\pm$ std)	Fields most often missing
Single-fault (small)	12	1.000 ± 0.000	—
Multi-fault (medium)	47	1.000 ± 0.000	—
Single-fault (large)	134	1.000 ± 0.000	—

Table 6.4: Notification structural completeness $\bar{S}$ per scenario (mean $\pm$ std over all notifications).

As reported in Table 6.4, a mean score of $\bar{S} = 1.0$ confirms that every notification carries sufficient context for an operator to comprehend the detected condition without consulting secondary data sources. SALLY ensures that if an event is published onto the event bus, the data in the event is static, thus a partial loss of data is not possible.

6.2.3 Level 3 Results: Projection Proxy

For each threshold-violation event, the test harness checks whether at least w preceding timesteps of the relevant measurement attribute were available in the data pipeline and visualized in Grafana (configured with $w = 100$ steps for all scenarios). The trend-data availability is the fraction of violation events for which the full history window was present.

Scenario	Violation Events	Full Window Available	Availability (%)
Single-fault (small)	8	8	100.0
Multi-fault (medium)	31	31	100.0
Single-fault (large)	89	86	96.6

Table 6.5: Trend-data availability per scenario (history window $w = 100$ steps).

Table 6.5 shows that full trend-data availability (100%) was achieved in the single-fault and multi-fault scenarios, confirming that the HDF5 provider delivers sequential timesteps without gaps and the GUI subscribes from the start of the replay. In the large scenario, three violation events occurred within the first 100 timesteps of the replay window, before a full history buffer had accumulated.

6.2.4 SA Proxy Scorecard

The scorecard in Table 6.6 provides a quantitative, reproducible approximation of SA support quality without requiring human subjects.

SA Level	Proxy Metric	Score
L1 – Perception	Display Completeness $C'(t)$	0.963
	SLD State Accuracy (%)	98.76
L2 – Comprehension	Notification Structural Completeness $\bar{S}$	1.000
L3 – Projection	Trend-Data Availability (%)	98.86

Table 6.6: Aggregated SA proxy scorecard across all tested configurations (mean values).

7 | Discussion

This chapter interprets the evaluation results presented in Chapter 6 in the context of the research questions and the central hypothesis of this thesis. It assesses which claims are supported by the evidence, identifies the limitations inherent in the evaluation approach, and discusses the implications for deploying SALLY in real-world SCADA environments.

7.1 INTERPRETATION OF RESULTS

The main research question (Section 1.1) asks how a notification and suggestion system can be designed and implemented to extract and present critical information, thereby enhancing operator SA in grid scenarios. The supportive research question (Section 1.1) concerns the considerations necessary for future integration into an existing SCADA system within a network control centre. This section interprets the evaluation results with respect to both questions and examines the extent to which the evidence supports the claim that SALLY contributes to operator SA.

Situational Awareness

The SA proxy scorecard (Table 6.6) provides the most direct quantitative evidence for the main research question by mapping system-level measurements onto the three levels of Endsley’s SA model. At Level 1 (perception), the display completeness of $C(t) = 0.963$ and the SLD state accuracy of 98.76% (Table 6.2 and ??) indicate that the data pipeline propagates virtually the entire grid state to the operator interface with faithful visual encoding, thereby satisfying the perception prerequisite of SA. At Level 2 (comprehension), the notification structural completeness of $\bar{S} = 1.000$ (Table 6.4) demonstrates that every emitted notification contains the full set of contextual fields required for an operator to interpret a detected condition without consulting secondary data sources. At Level 3 (projection), a trend-data availability of 98.86% (Table 6.5) confirms that the data pipeline retains sufficient historical context for operators to observe approaching violations before alarms are triggered. The residual shortfall in the latter metric is attributable to the tkinter-based SLD renderer rather than to the data backend or the Grafana visualisation, suggesting that the deficit is an implementation-level artefact rather than a systemic limitation.

While these scores establish that SALLY delivers the *informational prerequisites* for operator SA, Endsley’s model emphasises that SA is ultimately a cognitive phenomenon: operators must perceive, comprehend, and project grid states within their own mental model. Information availability is a necessary but not sufficient condition. SALLY addresses this cognitive dimension through two complementary mechanisms that extend beyond passive data delivery.

The first mechanism is deterministic scenario replay. Because the mosaic orchestrator can reproduce any co-simulation trajectory identically, trainers can construct repeatable training sessions that expose operators to specific fault patterns and grid events under controlled conditions. Operators can revisit the same scenario multiple times, progressively refining their ability to recognise relevant signals and distinguish critical deviations from normal operating behaviour. This form of deliberate, repeated exposure is closely aligned with the pattern-recognition processes that Endsley identifies as a primary driver of expert-level SA.

The second mechanism is the Rule Manager, which functions simultaneously as a *training instrument* and a *diagnostic tool* for SA. In a training context, trainees can be tasked with formulating rules that correctly detect particular faults in a given scenario. Constructing such a rule requires the trainee to identify the relevant entity and measurement attribute, select an appropriate comparison operator and threshold, and anticipate the conditions under which the rule should fire. This exercise maps directly onto Endsley’s Level 2 (comprehension), as the trainee must understand the causal structure of the data patterns rather than merely observe them. Successfully producing a rule that triggers on the correct fault condition constitutes observable evidence of comprehension-level SA. The Rule Manager equally supports a trainer-driven mode in which pre-configured rule sets generate notifications that guide trainees toward recognising conditions they might otherwise overlook, thereby scaffolding Level 1 (perception) and Level 3 (projection) skills by directing attention to relevant signals and approaching threshold violations.

The Grafana dashboards complement this training loop by offering an independent, time-series-level view of every metric and trace that SALLY produces. After a training run, both trainers and trainees can inspect the complete telemetry record against the co-simulation ground truth, closing the feedback cycle that is essential for sustained SA development.

Importantly, the training pathway and the proxy evaluation results reinforce one another. The high display completeness and SLD accuracy (Tables 6.2 and 6.3) ensure that the information operators train on is faithful to the ground truth, so that pattern-recognition skills acquired during training are transferable to equivalent real-world scenarios.

The notification structural completeness (Table 6.4) ensures that every notification encountered during training contains the full context needed for comprehension, thereby strengthening the link between rule triggers and their underlying causes. The near-complete trend-data availability (Table 6.5) enables operators to practise observing approaching violations in historical data, cultivating the anticipatory skills that Endsley’s Level 3 demands. Taken together, the quantitative proxy scores and the conceptual design of the training workflow provide converging evidence that Sally supports SA development across all three levels of Endsley’s model.

It is important to note what these proxy metrics can and cannot show. The metrics confirm that the system creates the *necessary informational preconditions* for operator SA: the correct data reaches the interface (display completeness), the visual encoding is faithful (SLD accuracy), notifications carry sufficient context (structural completeness), and trend history is available for anticipatory observation (trend-data availability). However, they do not and cannot confirm that an actual operator achieves improved perception, comprehension, or projection when using the system. SA is a cognitive phenomenon that depends on human attention, expertise, workload, and interface usability—factors that lie outside the scope of automated pipeline measurements. The proxy scores therefore represent an upper bound on the system’s *capacity* to support SA, not a measurement of SA itself. Bridging this gap requires the controlled human-subject studies discussed in the limitations and future work sections.

Functional Correctness and Scalability

The event bus benchmarks reported in Section 6.1.2 demonstrate that the core data transport sustains raw publishing throughput exceeding 95,000 events per second and end-to-end processing throughput of approximately 8,000 events per second with synchronous handlers. These figures exceed the throughput demands of typical SCADA telemetry by at least an order of magnitude, confirming that the architectural foundation of SALLY is sound with respect to the functional and performance requirements (FR5, NFR1). The benchmarks further validate that all four ring-buffer variants (SPSC, SPMC, MPSC, MPMC) sustain high operation rates under concurrent access patterns and that batch processing yields a 1.51x speed-up on the consumer side, satisfying the scalability requirements (NFR2) and indicating that the system can accommodate increased event rates as grid size or monitoring density grows.

The evaluation additionally tested scenarios of increasing complexity, ranging from a 24-entity single-fault grid to a 214-entity configuration. The resulting degradation was gradual rather than abrupt: display completeness decreased from 1.000 to 0.890, SLD accuracy from 100.0% to 96.5%, while notification completeness remained constant at 1.000. All observed deficits were confined to narrow bounds and traceable to specific implementation-level causes, in particular the rendering latency of the tkinter-based SLD under high entity counts. No systemic data loss or misrepresentation was detected, supporting NFR3's robustness claims. These results also highlight a clear avenue for future improvement: optimising or replacing the SLD renderer would likely eliminate the remaining accuracy gap for larger grid topologies.

In summary, the evaluation provides evidence that SALLY creates the informational prerequisites for all three SA levels while meeting its performance and scalability targets. It is important to note, however, that the proxy metrics measure the system's *capacity* to support SA rather than the cognitive SA of an actual operator. The implications of this distinction are examined in the following limitations section.

7.2 LIMITATIONS

As a bachelor's thesis conducted under constraints on time, resources, and scope, this work could not address every aspect of the system with equal depth—areas such as performant GUI rendering, comprehensive usability testing, and large-scale field deployment necessarily received less attention than the core architecture and evaluation. The following limitations must therefore be considered when interpreting the results and assessing their generalisability.

Absence of Human-Subject Validation. The most significant limitation is that SA was evaluated exclusively through automated proxy metrics. While these proxies, display completeness, SLD accuracy, notification structural completeness, and trend-data availability, quantify the system's capacity to *support* SA, they cannot confirm that actual operators achieve higher perception, comprehension, or projection when using the system. Subjective cognitive factors such as attentional tunnelling, trust calibration, alarm fatigue under sustained workload, and individual differences in expertise are not captured. A complete validation of the SA claims would require controlled user studies employing instruments such as SAGAT, or NASA-TLX with representative operator populations.

Simulated rather than Live Grid Data. All evaluation scenarios rely on deterministic replays of mosaik-generated co-simulation outputs stored in HDF5 files. While this ensures reproducibility and comparability, it does not capture the variability, noise, or communication failures that characterise live grid environments. Real-world data streams may arrive out of order, contain measurement errors, or exhibit latency profiles different from the deterministic replay used here. The system's behaviour under such conditions has not been empirically validated.

Scenario Coverage. Three scenario categories were tested: single-fault, multi-fault, and cascading failure. Although these represent a meaningful range of disturbance complexity, they do not exhaust the space of grid topologies, fault types, or operational contexts encountered in practice. In particular, scenarios involving very large grids (thousands of entities), simultaneous communication failures, or mixed-mode operation (partly live, partly replayed) remain untested.

GUI Evaluation Scope. The SA proxy evaluation focuses on the data layer, specifically whether the correct information reaches the GUI and whether notifications contain the required fields. It does not evaluate the visual design quality of the interfaces themselves, such as layout clarity, colour-contrast accessibility, or the cognitive effort required to parse the SLD under complex conditions. These aspects of HMI design are known to affect operator SA independently of data completeness.

Python Performance Ceiling. The system is implemented in Python, which imposes an inherent performance ceiling due to the Global Interpreter Lock (GIL) and interpreted execution. While the benchmarks demonstrate that throughput comfortably exceeds typical SCADA requirements, the scalability headroom for very-high-frequency monitoring scenarios (e.g. sub-second telemetry from thousands of smart meters) is bounded. The 11.5% throughput penalty observed for asynchronous handlers further illustrates that Python's concurrency model introduces overhead that would not exist in compiled-language implementations.

7.3 IMPLICATIONS FOR REAL-WORLD SCADA DEPLOYMENT

Transitioning SALLY from a laboratory prototype to a component within an operational SCADA environment raises a set of concrete technical challenges. This section examines the principal integration barriers that must be addressed.

Protocol Bridging and Data Latency

Production SCADA systems exchange measurements through industrial protocols such as IEC 60870-5-104 for telecontrol and IEC 61850 MMS for substation communication [71, 72]. SALLY's current data providers—HDF5 replay and TimescaleDB polling—do not speak these protocols natively. A deployment would therefore require a protocol-bridging adapter that translates incoming IEC frames into SALLY's internal `GridDataEvent` format and publishes them on the event bus. The latency budget for this translation is non-trivial: the event bus benchmarks show that the internal pipeline sustains sub-millisecond per-event overhead, but real-world network jitter, protocol handshake delays, and variable message sizes could dominate the end-to-end latency in ways that the deterministic replay evaluation did not capture. Quantifying this additional latency under realistic network conditions is a prerequisite for any deployment claim.

Architectural Enablers

Several architectural properties of SALLY facilitate future integration. The layered backend (NFR4) confines all data-source specifics to the infrastructure layer, so a new IEC 60870-5-104 adapter can be introduced without modifying domain logic or rule evaluation. The event bus's typed, topic-based publish-subscribe model maps naturally onto message-oriented middleware such as MQTT or AMQP, providing a standard interface point for external systems. The OTEL telemetry stack is compatible with the observability practices increasingly adopted in utility IT departments, and the YAML-based configuration model supports site-specific parameterisation without code changes.

Interoperability with Proprietary Platforms

The dominant SCADA platforms deployed by German distribution system operators, such as PSIconrol by PSI Software, are proprietary, closed-source systems whose internal architectures, API surfaces, and latency characteristics are not publicly documented. This opacity creates three specific integration risks. First, without knowledge of the host system's internal event loop and rendering pipeline, it is difficult to predict the end-to-end delay from the time a measurement arrives at the SCADA gateway until it appears on the operator's screen when Sally is inserted as an intermediary. Second, proprietary platforms may not expose extension points that allow a third-party module to inject notifications or overlay SLD highlights; in such cases, Sally would need to operate as a standalone companion application alongside the existing HMI rather than as an embedded extension. Third, vendor-specific data models and naming conventions may differ from the entity-attribute-value schema used by mosaik exports and Sally's internal domain model, requiring a non-trivial schema mapping layer.

8

Conclusion & Future work

8.1 CONCLUSION

To this end, the SALLY system was designed, implemented, and evaluated as a modular SCADA extension that ingests co-simulation data, evaluates user-defined rules over incoming grid measurements, generates notifications, and presents them through multiple coordinated GUI interfaces backed by a high-throughput event bus and a full OTEL-based observability stack.

The evaluation provides quantitative evidence that the system creates the informational prerequisites for all three levels of Endsley’s SA model. At Level 1 (perception), display completeness reached 0.963 and SLD state accuracy 98.76%, demonstrating that virtually all relevant grid state information is propagated to the operator interface with correct visual encoding. At Level 2 (comprehension), notification structural completeness of $\bar{S} = 1.0$ confirms that every notification carries the contextual fields needed to understand the meaning of a detected condition. At Level 3 (projection), trend-data availability of 98.86% shows that the data pipeline provides sufficient historical context for operators to observe approaching violations before they trigger alarms. On the performance side, the event bus sustains raw publishing throughput exceeding 95,000 events per second and end-to-end processing throughput of approximately 8,000 events per second, well above typical SCADA telemetry demands. Buffer benchmarks confirm scalability across all four concurrency patterns, and batch optimisation delivers a measurable 1.51x speed-up for consumer operations. These results satisfy the latency and throughput requirements defined in the system specification.

Beyond these quantitative scores, SALLY offers a conceptual pathway to improved SA through its combination of deterministic scenario replay and structured rule creation—a pathway that is plausible on theoretical grounds but has not been empirically validated in this thesis. Because mosaik-generated trajectories can be replayed identically, trainers could design targeted sessions in which operators repeatedly encounter specific fault patterns, building the pattern-recognition skills that Endsley identifies as central to experienced operator SA. The Rule Manager could reinforce this learning process by turning training into a measurable exercise: requiring trainees to formulate rules for a given scenario demands that they first understand the underlying data patterns, thresholds, and causal relationships, a task that maps directly to Endsley’s Level 2 (comprehension). Successfully creating a rule that triggers on the correct fault condition would constitute observable evidence of comprehension-level awareness.

Conversely, trainers could preconfigure rule sets that scaffold perception and projection skills by directing attention to relevant signals and approaching violations. The Grafana dashboards would close the feedback loop by providing an independent, time-series-level view of the full telemetry record against ground truth, allowing both trainers and trainees to verify whether the operator’s mental model matched reality. This training pathway aligns with the proxy evaluation results: the high display completeness and SLD accuracy ensure that the training data will be faithful, the perfect notification structural completeness guarantees rich contextual feedback, and the near-complete trend-data availability supports practising projection. While these proxy scores and the design of the training workflow provide *preliminary, mutually consistent* indications that Sally could support SA development across all three levels of Endsley’s model, confirming this hypothesis requires the controlled human-subject studies outlined in the future work.

Regarding future integration into an existing SCADA system within a network control centre (Section 1.1), the evaluation and discussion identify several enabling factors. The event bus’s typed domain events and publisher-subscriber model facilitate integration via standard protocols such as MQTT or HTTP. The OTEL telemetry stack aligns with emerging industry observability practices, and the layered backend architecture enables replacing replay-based data providers with live connectors without altering business logic. YAML-based configuration further supports site-specific adaptation. However, the transition to a live environment raises challenges that were not validated in this thesis, particularly data latency under real network conditions, protocol heterogeneity across vendor systems, and the closed ecosystems of proprietary SCADA platforms.

8.2 FUTURE WORK

The limitations identified in Chapter 7 delineate several areas in which the scope of this bachelor's thesis could not provide exhaustive treatment. The following directions outline how future work can build on the foundations established here, addressing the remaining constraints in evaluation depth, system maturity, and deployment readiness.

Human-Subject Validation of Situational Awareness

The most important next step is a controlled user study that validates whether the SA proxy improvements demonstrated in this thesis translate into measurable cognitive SA gains for human operators. Such a study should employ established instruments: SAGAT freezes to probe Level 1/2/3 awareness, NASA-TLX to assess workload, and SART for subjective SA ratings. The mosaik-based co-simulation environment is well-suited for this purpose, as scenarios can be frozen, replayed, and probed with identical timing. Comparing a group using SALLY's notifications against a baseline group using a conventional HMI would provide the statistical evidence that the proxy evaluation could not.

Training of new Employees

A primary use case for the system is training new and inexperienced grid operators. By linking the system to a co-simulated smart grid environment, SALLY can present trainees with scenarios that accurately reflect the modern power grid's operational complexity and unpredictability. The system can simulate critical events such as component disturbances or grid instability that could trigger cascading outages, which are difficult or unsafe to reproduce in real-world training. This approach supports the acquisition of both procedural knowledge and SA, which are essential for effective decision-making under pressure [73, 38].

Continuous Development for Experienced Operators

Beyond initial training, the system also supports continuous professional development for experienced personnel responsible for maintaining and securing the grid. SALLY provides a safe platform for operators of any experience level to practice responses to novel disturbances, test mitigation strategies, and refine their SA in the face of evolving grid dynamics. This continuous training could help maintain a high level of operational readiness and support knowledge transfer between experts and novices [73, 38].

Evaluation of Human-System Interaction in Digitalized Environments

The system permits systematic investigation of how digital features, such as pop-up notifications, interactive dashboards, and solution proposals, affect operator performance and cognitive workload. Integrated technologies for collecting both quantitative and qualitative grid data during simulation-based exercises enable assessment of whether these digital enhancements genuinely support operators or risk contributing to information overload and alarm fatigue. Such an assessment serves a dual purpose: first, the data could support the design of better human-machine interfaces; second, it could allow testing whether digitalization efforts yield real benefits in terms of operator performance and grid reliability [38].

Live Grid Data Integration

Replacing the HDF5 replay provider with a live data connector, for example via IEC 60870-5-104, IEC 61850 MMS, or MQTT bridging, would validate the system's behaviour under real-world data variability, out-of-order delivery, and communication failures. The layered architecture (NFR5) was designed to enable this transition without modifying core domain or rule-evaluation logic, but empirical validation remains outstanding.

Advanced Anomaly Detection

The current rule engine supports threshold-based and chained conditions. Future iterations could integrate machine-learning-based anomaly detection, such as LSTM forecasting or autoencoders trained on historical grid profiles, to complement rule-based detection with data-driven pattern recognition. The event bus architecture and dependency injection framework (NFR5) provide the extension points for plugging in additional detection providers alongside the existing rule engine.

Scalability to Large-Scale Grids

While the benchmarks confirm that the system exceeds typical SCADA throughput requirements, very-large-scale monitoring scenarios, such as those involving thousands of smart meters with sub-second telemetry, would benefit from a performance-critical reimplementation of the event bus core in a compiled language (e.g. Rust or C++) while retaining the Python-based application and presentation layers. This would address the Python performance ceiling identified in the limitations.

A | Appendix

A.1 DATABASE

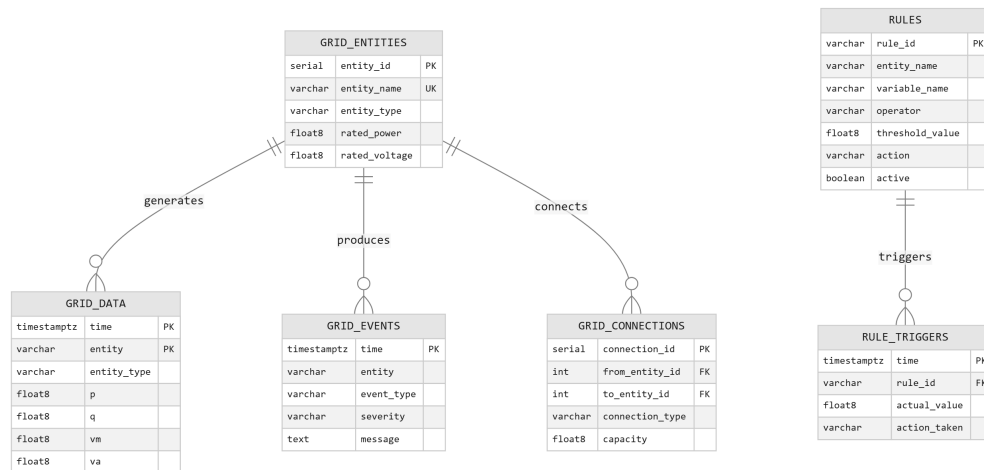


Figure A.1: ER-Diagram of TimescaleDB used in Sally

A.2 GRAFANA

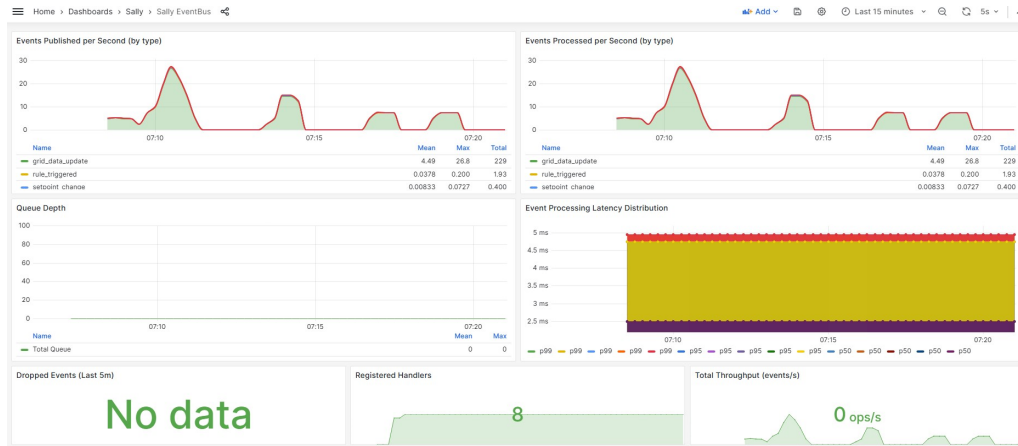


Figure A.2: Usage of the Event-Bus through Orchestrator and OTEL to review data in Grafana

Metric Naming Convention

All SALLY metrics follow a consistent Span naming convention with the `sally_` prefix:

```
bash
```

```
sally_{subsystem}_{metric_name}_{unit}
```

Examples:

```
sally_eventbus_events_published_total
sally_rules_evaluation_duration_ms
sally_scada_simulation_timestep
sally_setpoints_active_count
```

Codeblock-Listing A.1: Examples for Spans

TraceQL Queries

Grafana's Tempo datasource supports TraceQL for powerful trace filtering:

```
bash

# Find all Sally traces
{ resource.service.name =~ "SALLY.*" }

# Find slow operations (>100ms)
{ duration > 100ms }

# Find specific rule evaluations
{ name = "rules.evaluate" && span.rule_id = "voltage_high" }

# Find error traces
{ status = error }

# Find orchestrator commands
{ resource.service.name = "SALLY.Orchestrator"
  && name =~ "scada.command.*" }
```

Codeblock-Listing A.2: Example TraceQL Queries

Dashboard Panel Examples

```
bash

# Event rate by type (events per second)
rate(sally_eventbus_events_published_total[1m])

# P99 event processing latency
histogram_quantile(0.99, rate(sally_eventbus_event_latency_ms_bucket[5m]))

# Active rules count
sally_rules_active_count

# Current simulation timestep
sally_scada_simulation_timestep

# Rule trigger rate
rate(sally_rules_triggered_total[5m])
```

Codeblock-Listing A.3: Example Dashboard with Queries

Service Name Registry

Constant	Value	Component
ORCHESTRATOR	SAlly.Orchestrator	Main SCADA orchestration
EVENTBUS	SAlly.EventBus	High-performance event bus
RULES	SAlly.Rules	Rule evaluation engine
SETPOINTS	SAlly.Setpoints	Setpoint management
GRID_DATA	SAlly.GridData	Grid data collection
SERVICES	SAlly.Services	Async services runner
HDF5_BUILDER	SAlly.HDF5Builder	HDF5 file operations
MOSAİK_PARSER	SAlly.MosaikParser	Mosaik data parsing

Table A.1: Sally Service Names for Telemetry

Virtualization

Container	Use-Case	Ports
Grafana	Visualization of different Sally-Services	3000:3000
OTEL-Collector	Receives different types of data such as Metrics, Exports to Prometheus	4317:4317, 4318:4318, 8889:8889 (Prometheus Scrape)
Prometheus	Monitoring system with a dimensional data model, Datasource for Grafana	9090:9090
Tempo	Grafana Service for Traces and (complex) Querying & Data Exploration	3200:3200 (Querying), 28419:4317 (OTEL-GRPC)
Loki	Log aggregation system designed to store and query logs	3100:3100
Mosquitto	Mqtt-Service for integrating Code from OFFIS Guardian Code	1883:1883, 9001:9001
TimescaleDB	(Postgres) Database for Timeseries-Data using Hypertables and to store Entity-Relation-Data	5432:5432

A.3 PYTHON AND DEPENDENCIES

Technology Stack

Component	Library	Purpose
Widget Framework	tkinter	Core GUI widgets and event loop
Styling	ttkbootstrap	Modern themed widgets (Darkly, Solar, etc.)
Tables	Tableview	Sortable, filterable data tables
Dialogs	ttkbootstrap.dialogs	Message boxes, file dialogs
Canvas	tkinter.Canvas	SLD drawing and interaction

Table A.3: Frontend Technology Stack

Core Dependencies

Use-case	Dependency	Reasoning	Alternatives
Async runtime	asyncio	Python standard async library, used in all async services	–
Alternative async runtime	trio	Structured concurrency with better error handling	curio
Async-Trio bridge	trio-asyncio	Enables mixing asyncio and trio code	–
Lightweight threading	greenlet	Lightweight pseudo-concurrent programming for gevent	threading
Async networking	gevent	Greenlet-based networking library	asyncio

Data Handling

Use-case	Dependency	Reasoning	Alternatives
HDF5 tables	h5py	HDF5-Support & table operations	tables (PyTables)
Numerical computing	numpy	Array operations, used throughout simulation	cupy

Use-case	Dependency	Reasoning	Alternatives
Data manipulation	pandas	DataFrame operations for time-series data	polars
Fast DataFrames	polars	High-performance alternative for large datasets	pandas

Database & Persistence

Use-case	Dependency	Reasoning	Alternatives
Async PostgreSQL driver	asyncpg	Fast async PostgreSQL/TimescaleDB driver	psycopg2 (sync)
Async HTTP client/server	aiohttp	Mature async HTTP for API and WebSocket	httpx

Co-Simulation (Mosaik Ecosystem)

Use-case	Dependency	Reasoning	Alternatives
Co-simulation framework	mosaik	Smart grid co-simulation orchestration	–
Mosaik simulator API	mosaik-api	Build custom mosaik simulators	–
Power flow simulation	pandapower	Power system modeling and analysis	GridLAB-D
Network graphs	networkx	Grid topology representation	igraph
Discrete event simulation	simpy	Event-driven simulation support	–

MIDAS Integration

Use-case	Dependency	Reasoning	Alternatives
MIDAS core	midas-mosaik	MIDAS mosaik integration	–

Use-case	Dependency	Reasoning	Alternatives
Power grid scenarios	<code>midas-powergrid</code>	Power grid scenario generation	–
MIDAS utilities	<code>midas-util</code>	Common MIDAS utilities	–

Configuration & DI

Use-case	Dependency	Reasoning	Alternatives
YAML parsing	<code>pyyaml</code>	Configuration file parsing	<code>toml</code>
Dependency injection	<code>dependency-injector</code>	IoC container for service wiring	<code>injector</code>
Environment config	<code>python-decouple</code>	Environment variable management	<code>python-dotenv</code>
Data validation	<code>pydantic</code>	Runtime data validation and settings	<code>attrs</code>
Data classes	<code>attrs</code>	Lightweight dataclass alternative	<code>dataclasses</code>

Logging & Observability

Use-case	Dependency	Reasoning	Alternatives
Structured logging	<code>loguru</code>	Modern logging with rotation	<code>logging</code>
Log formatting	<code>structlog</code>	Structured log output	<code>python-json-logger</code>
Telemetry SDK	<code>otel-sdk</code>	Distributed tracing and metrics	–
OTLP export	<code>otel-exporter-otlp</code>	Export to OTEL Collector	–

GUI (Desktop)

Use-case	Dependency	Reasoning	Alternatives
Modern Tkinter themes	<code>ttkbootstrap</code>	Bootstrap-styled Tkinter widgets	<code>customtkinter</code>
Custom widgets	<code>customtkinter</code>	Modern-looking Tkinter widgets	<code>ttkbootstrap</code>

Use-case	Dependency	Reasoning	Alternatives
Image handling	pillow	Image loading for GUI icons	–

Web Frontend

Use-case	Dependency	Reasoning	Alternatives
Web framework	django	Web application framework	fastapi
REST API	django restframework	REST API endpoints	–
WebSocket	channels	Django WebSocket support	–
ASGI server	daphne	ASGI server for channels	uvicorn

SCADA Web Bridge

Use-case	Dependency	Reasoning	Alternatives
MQTT client	paho-mqtt	MQTT messaging for IoT/SCADA	aiomqtt
WebSocket server	python-socketio	Socket.IO for real-time web	websockets

Development

Use-case	Dependency	Reasoning	Alternatives
Testing	pytest	Test framework	unittest
Async testing	pytest-asyncio	Async test support	–
Formatting	black	Code formatter	autopep8
Linting	ruff	Fast Python linter	flake8
Type checking	mypy	Static type checking	pyright

Notes on Dependencies

- **h5py vs PyTables:** h5py is used for direct HDF5 access; PyTables (**tables**) provides higher-level table abstractions

- **pandas vs polars:** pandas is used for compatibility; polars is available for performance-critical paths
- **asyncio vs trio:** Sally primarily uses asyncio but supports trio via trio-asyncio for structured concurrency
- **ttkbootstrap:** Provides modern Bootstrap-styled themes for Tkinter without requiring a web stack

A.4 EVENT-BUS

Handler Protocols

The bus supports both synchronous and asynchronous handler protocols for flexibility:

```
python
class EventHandler(Protocol):
    """Async handler protocol for complex processing."""
    @property
    def event_types(self) -> List[str]:
        """Event types this handler subscribes to."""
        ...
    async def handle(self, event: Event) -> None:
        """Handle event asynchronously."""
        ...
class SyncEventHandler(Protocol):
    """Sync handler protocol for maximum throughput."""
    @property
    def event_types(self) -> List[str]:
        ...
    def handle_sync(self, event: Event) -> None:
        """Handle event synchronously (faster)."""
        ...
```

Codeblock-Listing A.4: Event Handler Protocols

Event Types and Domain Events

Sally defines a set of domain events that flow through the event bus, enabling loose coupling between components.

Event Class	Purpose
GridDataEvent	Carries measurement updates from simulation step
ControlActionEvent	Represents control commands to grid entities
RuleTriggeredEvent	Signals that a rule condition was met
SimulationStepEvent	Indicates simulation timestep advancement
SimulationStateEvent	Broadcasts current simulation state
SetpointChangeEvent	Notifies setpoint modifications

Table A.15: Domain Event Types

Core Event Class

```
python
@dataclass(slots=True)
class Event:
    """Base event class optimized for high-throughput.

    Uses __slots__ for memory efficiency and faster
    attribute access.
    """
    event_type: str
    timestamp: float = field(default_factory=time.time)
    correlation_id: Optional[str] = None
    metadata: Dict[str, Any] = field(default_factory=dict)
```

Codeblock-Listing A.5: Base Event Class with Slots Optimization

List of Figures

2.1	Smart Grid Three-Layer Architecture Model showing the integration of physical infrastructure, communication networks, and application services. Source: [16]	4
2.2	Typical SCADA system architecture organised in five hierarchical layers, from field devices at the bottom to the central monitoring station at the top [23].	6
2.3	Illustration of a SCADA HMI screen showing a SLD with component labels and associated values [34].	11
4.1	Rule Manager GUI Layout	29
4.2	SCADA Display Layout	30
4.3	Sally System Architecture Overview	32
5.1	Sally Event Bus architecture	35
5.2	GPO-Stack Telemetry Data Flow Architecture	43
5.3	Sally SCADA & Simulation	44
5.4	SALLY backend layered architecture. Solid arrows denote direct downward dependencies; dashed arrows indicate cross-cutting access to the Core layer, which provides shared infrastructure (event bus, logging, configuration, telemetry) to all layers above.	46
5.5	Entity-relationship diagram of the TimescaleDB schema.	50
5.6	SCADA Orchestrator Simulation Step Sequence	52
5.7	Frontend Component Architecture	54
5.8	Rule-Manager-GUI	55
5.9	Rule-Manager GUI where rule(s) got triggered	56
5.10	Basic Rule Creation	57
5.11	Rule Chain Example	57
5.12	SLD visualization with Setpoint-Window and triggered rules in current simulation step	58
5.13	Rule Chain Example	59
6.1	Push & Pop Comparison between all Buffers.	61
6.2	Buffer-Lock overhead comparison.	62
6.3	End-to-End Processing Throughput comparison between Synchronous and Asynchronous Handlers.	63

A.1	ER-Diagram of TimescaleDB used in SAlly	80
A.2	Usage of the Event-Bus through Orchestrator and OTEL to review data in Grafana	81

List of Tables

5.1	SAlly mosaik simulators and their roles [65].	40
5.2	SAlly Simulation Modes.	41
5.3	Pre-configured Grafana Dashboards	43
5.4	Comparison Operators	56
6.1	Single-Thread Push/Pop Performance (500K operations)	61
6.2	Display completeness ratio $C(t)$ across scenarios (mean $\pm$ std over all timesteps).	65
6.3	SLD state accuracy (%) by scenario.	65
6.4	Notification structural completeness $\bar{S}$ per scenario (mean $\pm$ std over all notifications).	66
6.5	Trend-data availability per scenario (history window $w = 100$ steps).	66
6.6	Aggregated SA proxy scorecard across all tested configurations (mean values).	67
A.1	SAlly Service Names for Telemetry	83
A.3	Frontend Technology Stack	84
A.15	Domain Event Types	90

Bibliography

- [1] Sandra Venghaus, Meike Henseleit, and Maria Belka. “The impact of climate change awareness on behavioral changes in Germany: changing minds or changing behavior?” In: *Energy, Sustainability and Society* 12.1 (Feb. 9, 2022), p. 8. ISSN: 2192-0567. DOI: 10.1186/s13705-022-00334-8. URL: <https://doi.org/10.1186/s13705-022-00334-8>.
- [2] Catriona McKinnon. *Climate change and future justice: Precaution, compensation and triage*. Routledge, 2012. URL: <https://www.academia.edu/download/48469123/PROOFS.pdf>.
- [3] Bundesministerium für Wirtschaft und Energie (BMWE). *Energiewende. Effizient. Machen*. 2025. URL: https://www.bundeswirtschaftsministerium.de/Redaktion/DE/Publikationen/Energie/energiewende-effizient-machen.pdf?__blob=publicationFile&v=20.
- [4] Bundes-Klimaschutzgesetz (KSG). “Klimaschutzgesetz 2021 (KSG)”. In: *Bundes-Klimaschutzgesetz* (2021).
- [5] Federal Ministry for Economic Affairs and Energy (BMWi). *Smart Grid Roadmap 2030*. Digital grid transition strategy. 2017. URL: <https://www.bmwk.de/Redaktion/EN/Dossier/grids-grid-expansion.html>.
- [6] Messstellenbetriebsgesetz. “Messstellenbetriebsgesetz”. In: *Messstellenbetriebsgesetz* (2023).
- [7] Chibuike Peter Ohanu, Salihu Ahmed Rufai, and Ugbe Christiana Oluchi. “A comprehensive review of recent developments in smart grid through renewable energy resources integration”. In: *Heliyon* 10.3 (2024), e25705. ISSN: 2405-8440. DOI: <https://doi.org/10.1016/j.heliyon.2024.e25705>. URL: <https://www.sciencedirect.com/science/article/pii/S2405844024017365>.
- [8] Seref Sagiroglu et al. “Big data issues in smart grid systems”. In: *2016 IEEE International Conference on Renewable Energy Research and Applications (ICRERA)*. 2016, pp. 1007–1012. DOI: 10.1109/ICRERA.2016.7884486.
- [9] ICS Investigation Expert Panel. *Grid Incident in Spain and Portugal on 28 April 2025*. Tech. rep. ICS Investigation Expert Panel, 2025. URL: [https://eepublicdownloads.blob.core.windows.net/public-cdn-container/clean-documents/Publications/2025/entso-e_incident_report_ES-PT_April_2025_06.pdf%20\(Link%20is%20broken%20for%20some%20reason,%20Google%20result%20still%20points%20to%20the%20link%20if%20the%20title%20is%20googled.\)](https://eepublicdownloads.blob.core.windows.net/public-cdn-container/clean-documents/Publications/2025/entso-e_incident_report_ES-PT_April_2025_06.pdf%20(Link%20is%20broken%20for%20some%20reason,%20Google%20result%20still%20points%20to%20the%20link%20if%20the%20title%20is%20googled.)).

- [10] Federal Ministry for Economic Affairs and Climate Action. “Smart Grids”. In: *Federal Ministry for Economic Affairs and Climate Action* (n.d.). URL: <https://www.bundeswirtschaftsministerium.de/Redaktion/EN/Artikel/Energy/smart-grids.html> (visited on 03/09/2026).
- [11] Dejan Ilic, PG Da Silva, and Stamatis Karnouskos. “The Impact of Smart Grid Prosumer Grouping on Forecasting Accuracy and Its Benefits for Local Electricity Market Trading”. In: *IEEE Transactions on Smart Grid* 8.2 (2014), pp. 402–410. URL: <https://ieeexplore.ieee.org/document/6691297>.
- [12] Thomas Strasser et al. “A review of architectures and concepts for intelligence in future electric energy systems”. In: *IEEE Transactions on Industrial Electronics* 62.4 (2014), pp. 2424–2438. URL: <https://ieeexplore.ieee.org/document/6912080>.
- [13] M Riduan Abid et al. “Intelligent building control systems for thermal comfort and energy-efficiency: A systematic review of artificial intelligence-assisted techniques”. In: *Renewable and Sustainable Energy Reviews* 144 (2021), p. 110969. URL: <https://doi.org/10.1016/j.rser.2021.110969>.
- [14] Hyosik Yang and Leila Tightiz. “A comprehensive review on IoT protocols’ features in smart grid communication”. In: *Energies* 13.11 (2020), p. 2762. URL: <https://doi.org/10.3390/en13112762>.
- [15] Jesús Rodríguez-Molina et al. “Business models in the smart grid: Challenges, opportunities and proposals for prosumer profitability”. In: *Energies* 7.9 (2014), pp. 6142–6171. URL: <https://doi.org/10.3390/en7096142>.
- [16] 2024 Adila El Maghraoui et. al. “Advancements in Technology for Renewable Energy Microgrids”. In: (2024). URL: https://www.researchgate.net/publication/383123301_Advancements_in_Technology_for_Renewable_Energy_Microgrids/figures?lo=1.
- [17] Lexuan Meng et al. “Review on control of DC microgrids and multiple microgrid clusters”. In: *IEEE journal of emerging and selected topics in power electronics* 5.3 (2017). URL: <https://ieeexplore.ieee.org/document/7890986>.
- [18] Nikos D. Hatziargyriou et al. “Microgrids – Large Scale Integration of Microgeneration to Low Voltage Grids”. In: *CIGRE 2006*. hal-00526633. Conseil International des Grands Réseaux Électriques. Paris, France, Aug. 2006. URL: <https://minesparis-psl.hal.science/hal-00526633/document>.

- [19] Miljenko Mikuc, Mario Sverko, and Tihana Galinac Grbac. “Scada systems with focus on continuous manufacturing and steel industry: A survey on architectures, standards, challenges and industry 5.0”. In: *IEEE access* 10 (2022), pp. 109395–109430. URL: <https://ieeexplore.ieee.org/document/9907002>.
- [20] Anya Heider et al. “Flexibility options and their representation in open energy modelling tools”. In: *Energy Strategy Reviews* 38 (2021), p. 100737. URL: <https://doi.org/10.1016/j.esr.2021.100737>.
- [21] David Bailey and Edwin Wright. *Practical SCADA for industry*. Elsevier, 2003.
- [22] Mini S Thomas and John Douglas McDonald. *Power system SCADA and smart grids*. CRC press, 2017.
- [23] Sherif Abdelwahed Qian Chen. “Towards realizing self-protecting SCADA systems”. In: (2014). URL: https://www.researchgate.net/publication/266658794_Towards_realizing_self-protecting_SCADA_systems/figures?lo=1.
- [24] Vinay M Ijure, Sean A Laughter, and Ronald D Williams. “Security issues in SCADA networks”. In: *computers & security* 25.7 (2006), pp. 498–506.
- [25] Mica R Endsley. “Toward a theory of situation awareness in dynamic systems”. In: *Situational awareness*. Routledge, 2017, pp. 9–42.
- [26] Ross Guttromson et al. “Human Factors for Situation Assessment in Grid Operations”. In: *Human Factors for Situation Assessment in Grid Operations*. Jan. 2007. DOI: 10.2172/935904. URL: https://www.pnnl.gov/main/publications/external/technical_reports/PNNL-16780.pdf.
- [27] Susan Stevens-Adams et al. “Situation Awareness and Automation in the Electric Grid Control Room”. In: *Procedia Manufacturing* 3 (2015). 6th International Conference on Applied Human Factors and Ergonomics (AHFE 2015) and the Affiliated Conferences, AHFE 2015, pp. 5277–5284. ISSN: 2351-9789. DOI: <https://doi.org/10.1016/j.promfg.2015.07.609>. URL: <https://www.sciencedirect.com/science/article/pii/S2351978915006101>.
- [28] Chao Gong. “Human-machine interface: Design principles of visual information in human-machine interface design”. In: *2009 International Conference on Intelligent Human-Machine Systems and Cybernetics*. Vol. 2. IEEE. 2009, pp. 262–265.
- [29] Friedhelm Nachreiner, Peter Nickel, and Inga Meyer. “Human factors in process control systems: The design of human-machine interfaces”. In: *Safety Science* 44.1 (2006), pp. 5–26.

- [30] Edge Zero. *Situational Awareness Solutions*. 2023. URL: <https://edgezero.co/solutions/situational-awareness-solutions/>.
- [31] Charles M Adrah et al. “Experimental validation of a new impedance-based protection for networks with distributed generation using co-simulation test platform”. In: *IEEE Transactions on Power Delivery*. Vol. 35. 3. IEEE. 2019, pp. 1136–1145. URL: <https://ieeexplore.ieee.org/document/8805131>.
- [32] Saeed Salah, Gabriel Macía-Fernández, and Jesús E Díaz-Verdejo. “A model-based survey of alert correlation techniques”. In: *Computer Networks* 57.5 (2013), pp. 1289–1317. URL: <https://www.sciencedirect.com/science/article/abs/pii/S1389128612004124>.
- [33] Jean-Yves Fiset. *Human-machine interface design for process control applications*. ISA, 2009.
- [34] “Scalink Supervisor”. In: (2020). URL: <https://www.sael.it/en/paper-automation-section/scalink-supervisor/>.
- [35] EFEN GmbH. *Smart Grid Interface Module*. 2022. URL: https://www.efen.com/assets/downloads/Broschueren-EN/EFEN_Smart_Grid_Interface_EN.pdf.
- [36] Abdun Mahmood, Carlos Queiroz, and Zahir Tari. “SCADASim-A framework for building SCADA simulations”. In: *IEEE Transactions on Smart Grid*. Vol. 2. 4. 2011, pp. 589–597. DOI: 10.1109/TSG.2011.2164814. URL: https://www.researchgate.net/publication/220592786_SCADASim_-_a_framework_for_building_SCADA_simulations#fullTextFileContent.
- [37] Bernardi Pranggono and Martin Annor-Asante. “Development of smart grid testbed with low-cost hardware and software for cybersecurity research”. In: *Wireless Personal Communications*. Vol. 101. 2018, pp. 1357–1377. DOI: 10.1007/s11277-018-5769-3. URL: https://www.researchgate.net/publication/324711612_Development_of_Smart_Grid_Testbed_with_Low-Cost_Hardware_and_Software_for_Cybersecurity_Research_and_Education.
- [38] Lena Engelmann, Arlena Wellßow, and Eric Veith. “SHOW-OFF - Towards a Framework for Comprehensive and Systematic Visual Analysis of Learning Agents Performance in Smart Grid Co-Simulations”. In: *SHOW-OFF - Towards a Framework for Comprehensive and Systematic Visual Analysis of Learning Agents Performance in Smart Grid Co-Simulations*. 2024.
- [39] Cláudio Gomes et al. “Co-Simulation: A Systematic Literature Review”. In: *ACM Computing Surveys* 51.3 (2018), pp. 1–33. DOI: 10.1145/3179993.

- [40] Peter Palensky et al. “Cosimulation of Intelligent Power Systems: Fundamentals, Software Architecture, Numerics, and Coupling”. In: *IEEE Industrial Electronics Magazine* 11.1 (2017), pp. 34–50. DOI: 10.1109/MIE.2016.2639825.
- [41] Cornelius Steinbrink et al. “CPES Testing with mosaik: Co-Simulation Planning, Execution and Analysis”. In: *Applied Sciences* 9.5 (2019), p. 923. DOI: 10.3390/app9050923.
- [42] Poria Astero et al. “Smart Grid Co-Simulation by Developing an FMI-Compliant Interface for PSCAD”. In: *CIREN 2019 Conference*. 2019, p. 849. DOI: 10.34890/330. URL: <https://scispace.com/pdf/smart-grid-co-simulation-by-developing-an-fmi-compliant-16w9apr8j5.pdf>.
- [43] Rui Yang, Michael Ingram, and Mengmeng Cai. *Situational Awareness of Grid Anomalies (SAGA)*. Tech. rep. National Renewable Energy Laboratory, 2022. URL: <https://www.nrel.gov/docs/fy23osti/84465.pdf>.
- [44] Mingzhi Zhang et al. “An asynchronous real-time co-simulation platform for modeling interaction between microgrids and power distribution systems”. In: *2019 IEEE Power & Energy Society General Meeting (PESGM)*. IEEE. 2019, pp. 1–5. URL: <https://ieeexplore.ieee.org/document/8973802>.
- [45] Anna Förster et al. “OMNeT++ and mosaik: enabling simulation of Smart Grid communications”. In: *arXiv preprint arXiv:1509.03067*. 2015. URL: <https://arxiv.org/abs/1509.03067>.
- [46] Antonello Monti et al. “A Global Real-Time Superlab: Enabling High Penetration of Power Electronics in the Electric Grid”. In: *IEEE Power Electronics Magazine* 5.3 (2018), pp. 35–44. DOI: 10.1109/MPEL.2018.2850698.
- [47] Python Software Foundation. *Python Programming Language*. <https://www.python.org>. Accessed: 2026-02-17. 2025.
- [48] The HDF Group. *Official HDF5 Library Repository*. <https://github.com/HDFGroup/hdf5>. GitHub repository, accessed: 2026-02-17. 2025.
- [49] The HDF Group. *HDF5: High-performance data management and storage suite*. <https://www.hdfgroup.org/solutions/hdf5/>. Accessed: 2026-02-17. 2025.
- [50] Andrew Collette and contributors. *HDF5 for Python*. <https://docs.h5py.org/en/stable/>. h5py documentation, accessed: 2026-02-17. 2025.
- [51] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003. ISBN: 978-0321200686.

- [52] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN: 978-0201633610.
- [53] Patrick Th. Eugster et al. “The Many Faces of Publish/Subscribe”. In: *ACM Computing Surveys* 35.2 (2003), pp. 114–131. DOI: 10.1145/857076.857078.
- [54] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. MIT Press, 2009. ISBN: 978-0262033848.
- [55] Henry S. Warren. *Hacker’s Delight*. 2nd. Addison-Wesley, 2012. ISBN: 978-0321842688.
- [56] Martin Thompson et al. *LMAX Disruptor: High Performance Inter-Thread Messaging Library*. <https://lmax-exchange.github.io/disruptor/>. Accessed: 2026-02-24. 2011.
- [57] Leslie Lamport. “Specifying Concurrent Program Modules”. In: *ACM Transactions on Programming Languages and Systems* 5.2 (1983). Introduced the single-producer single-consumer bounded buffer with sequence-number indices, foundational to modern lock-free ring buffers., pp. 190–222. DOI: 10.1145/69624.357207.
- [58] Maged M. Michael and Michael L. Scott. “Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms”. In: *Proceedings of the 15th Annual ACM Symposium on Principles of Distributed Computing (PODC)*. ACM, 1996, pp. 267–275. DOI: 10.1145/248052.248106.
- [59] David R. Butenhof. *Programming with POSIX Threads*. Addison-Wesley, 1997. ISBN: 978-0201633924.
- [60] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. Revised 1st. Morgan Kaufmann, 2012. ISBN: 978-0123973375.
- [61] Timescale Inc. “TimescaleDB Documentation”. In: *Timescale Inc.* (2025). Accessed: 2026-02-17. URL: <https://docs.timescale.com/>.
- [62] Timescale Inc. “Tiger Data: PostgreSQL++ for Time Series, Analytics AI”. In: *Timescale Inc.* (2025). Accessed: 2026-02-17. URL: <https://www.timescale.com/>.
- [63] Grafana Labs. “Grafana Documentation”. In: *Grafana Labs* (2025). Accessed: 2026-02-17. URL: <https://grafana.com/docs/>.
- [64] Anders Beck et al. “Spacio-temporal situation assessment for mobile robots”. In: *Researchgate* (Jan. 2011). URL: https://www.researchgate.net/publication/252027632_Spacio-temporal_situation_assessment_for_mobile_robots.
- [65] Philip Weber. *Sally — SCADA Extension for Operator Situational Awareness in Smart Grid Environments*. GitHub repository. Accessed: 2026-03-20. 2026. URL: <https://github.com/flipflopsen/Sally>.

- [66] Python Software Foundation. *asyncio — Asynchronous I/O*. <https://docs.python.org/3/library/asyncio.html>. Accessed: 2026-02-24. 2025.
- [67] Cornelius Steinbrink et al. “CPES Testing with mosaik: Co-Simulation Planning, Execution and Analysis”. In: *Applied Sciences* 9.5 (2019). ISSN: 2076-3417. DOI: 10.3390/app9050923. URL: <https://www.mdpi.com/2076-3417/9/5/923>.
- [68] OFFIS. *mosaik Documentation*. <https://mosaik.readthedocs.io/en/latest/index.html>. A flexible smart-grid co-simulation framework documentation. 2025. (Visited on 09/16/2025).
- [69] OFFIS. *mosaik: A Flexible Smart Grid Co-simulation Framework*. <https://gitlab.com/mosaik>. GitLab repository for mosaik smart grid co-simulation framework. OFFIS, 2025. (Visited on 09/16/2025).
- [70] MIDAS Development Team. *MIDAS-Mosaik Energy Simulation Package*. Accessed: 2024-12-18. 2024. URL: <https://pypi.org/project/midas-mosaik/>.
- [71] ipcomm. “IEC 60870-5-104”. In: *International Electrotechnical Commission (IEC)* (2026). URL: <https://www.ipcomm.de/protocol/IEC104/de/sheet.html>.
- [72] Christoph Brunner. “IEC 61850 for power system communication”. In: *2008 IEEE/PES Transmission and Distribution Conference and Exposition*. 2008, pp. 1–6. DOI: 10.1109/TDC.2008.4517287.
- [73] Arlena Wellßow, Torben Logemann, and Eric MSP Veith. “Trajectory Generation Model: Building a Simulation Link Between Expert Knowledge and Offline Learning”. In: *Proceedings of the 14th International Conference on Simulation and Modeling Methodologies, Technologies and Applications (SIMULTECH 2024)*. Paris, France: SCITEPRESS – Science and Technology Publications, Lda., 2024, pp. 91–102. ISBN: 978-989-758-708-5. DOI: 10.5220/0012764900003758. URL: <https://www.scitepress.org/Papers/2024/127649/127649.pdf>.

Usage of Generative AI

Generative AI tools were used during the preparation of this thesis. Their use was limited to supportive tasks; all generated outputs were reviewed, verified, and, where necessary, revised by the author. The tools and their respective roles are listed below.

Perplexity (Deep Research mode) was used for exploratory literature searches and for obtaining concise summaries of research papers during the early stages of the literature review. All sources identified through Perplexity were independently verified and cited from their original publications.

GitHub Copilot (Claude Sonnet 4 / Claude Opus 4) was used as an integrated coding assistant during software development. Its contributions include boilerplate code generation, routine programming tasks, unit and integration test scaffolding, and support for iterative debugging. Additionally, Copilot was used in agent mode to assist with drafting, restructuring, and refining sections of the thesis text, as well as generating feedback. All code and text produced with Copilot were manually reviewed and adapted.

Grammarly was used for proofreading written content, including grammar and spelling checks, as well as suggestions for improving clarity, sentence structure, and vocabulary.

No generative AI tool was used to produce the concluded results, draw conclusions, or make design decisions. The responsibility for the correctness and integrity of all content remains with the author.

Eidesstattliche Versicherung

Hiermit versichere ich an Eides statt, dass ich diese Arbeit selbstständig verfasst habe und keine anderen als die angegebenen Quellen verwendet, sowie keine unzulässige fremde Hilfe in Anspruch genommen habe. Zudem wurde die Arbeit nicht unter unkenntlichem Einsatz generativer KI erbracht. Außer- dem versichere ich, dass ich die allgemeinen Prinzipien wissenschaftlichen Arbeitens und Veröffentlichens, wie sie in den Leitlinien guter wissenschaftlicher Praxis der Carl von Ossietzky Universität Oldenburg festgelegt sind, befolgt habe. Weiterhin erkläre ich, dass die Arbeit in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt wurde.

Mir ist bekannt, dass eine eidesstattliche Versicherung eine nach den §§ 156, 161 Strafgesetzbuch (StGB) strafbewertete Bestätigung der Richtigkeit meiner Erklärung ist. Mir sind die strafrechtlichen Folgen einer unrichtigen, d.h. nicht den Tatsachen entsprechenden und unvollständigen Erklärung, d. h. das Verschweigen der wesentlichen Tatsachen bekannt.